Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Операционные среды и системное программирование

К защите допустить:
И.О. Заведующего кафедрой информатики
_____ С. И. Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

РАЗРАБОТКА КОНСОЛЬНОГО НАВИГАТОРА ПО ФАЙЛОВОЙ СИСТЕМЕ С ДРЕВОВИДНЫМ ОТОБРАЖЕНИЕМ КАТАЛОГОВ

БГУИР КП 6-05-0612-02 012 ПЗ

Студент А. И. Кохан

Руководитель Н. Ю. Гриценко

Минск 2026

СОДЕРЖАНИЕ

Введение 5
1 Архитектура программного обеспечения 6
1.1 Архитектура TUI-приложений 6
1.2 Файловая система Unix 6
1.3 Модель представления файловой системы в памяти 8
2 Платформа программного обеспечения 10
2.1 Библиотеки управления терминалом 10
2.2 Системные API файловой системы 11
3 Теоретическое обоснование разработки программного продукта 13
3.1 Обоснование необходимости разработки 13
3.2 Иерархическая структура файловых систем 19
3.3 Права доступа в Unix/Windows 20
4 Проектирование функциональных возможностей программы 23
4.1 Описание обработки ввода пользователя 23
4.2 Описание логики отрисовки дерева 27
4.3 Описание реализации файловых операций 27
4.4 Тестирование программы 28
5 Архитектура разрабатываемой программы 31
5.1 Общая структура программы 31
5.2 Описание функциональной схемы навигатора 32
5.3 Описание блок схемы обработки событий ввода и перерисовки 33
Заключение 35
Список литературных источников 36
Приложение А (обязательное) Листинг программного кода 37
Приложение Б (обязательное) Функциональная схема навигатора 47
Приложение В (обязательное) Блок схема алгоритма, обработки событий ввода и перерисовки экрана 48
Приложение Г (обязательное) Графический интерфейс пользователя 49
Приложение Д (обязательное) Ведомость курсового проекта 50

ВВЕДЕНИЕ

В ранних Unix-системах взаимодействие с файловой системой осуществлялось исключительно через командную строку с помощью таких утилит, как ls, cp, mv, rm. С ростом сложности задач и необходимостью более наглядного представления структуры каталогов появились консольные файловые менеджеры, например, Midnight Commander [1], который предоставил двухпанельный интерфейс прямо в терминале. Эти программы стали незаменимым инструментом для системных администраторов и разработчиков, работающих в среде Linux и других Unix-подобных операционных системах.

В современных условиях, несмотря на широкое распространение графических рабочих окружений, роль консольных инструментов в экосистеме Linux остаётся высокой. Администрирование серверов, разработка встраиваемых систем и контейнерная виртуализация часто не предполагают наличия графического интерфейса. В таких сценариях консольные навигаторы (mc, ranger, vifm) обеспечивают эффективное перемещение по файловой системе, быстрое выполнение операций над файлами и просмотр их содержимого, оставаясь при этом легковесными и полностью управляемыми с клавиатуры [2].

Актуальность данного проекта заключается в глубоком понимании механизмов взаимодействия пользовательских приложений с ядром Linux на уровне системных вызовов. Разработка собственного консольного навигатора позволяет на практике изучить работу с файловым вводом-выводом через системные вызовы. Кроме того, реализация текстового интерфейса требует знаний о низкоуровневом управлении терминалом через termios или с использованием библиотеки ncurses, в области системного программирования под Linux.

Целью курсового проекта является разработка интерактивного консольного приложения – навигатора по файловой системе с древовидным отображением каталогов, ориентированного на работу в среде Linux. Программа должна предоставлять пользователю возможность удобной навигации по иерархической структуре файловой системы и выполнять базовые операции (просмотр, копирование, перемещение, удаление, создание файлов и каталогов) с использованием системных вызовов Linux. Также, требованием является корректная обработка всех возможных ошибок, связанных с правами доступа и состоянием файловой системы, с выводом информативных сообщений.

1 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1.1 Архитектура TUI-приложений

При разработке консольного навигатора для операционной системы Linux архитектура программного обеспечения строилась с учетом особенностей системных вызовов и файловой модели Unix-подобных систем.

Текстовый интерфейс пользователя (TUI) занимает промежуточное положение между графическими интерфейсами и чисто командной строкой. В отличие от графических приложений, которые отрисовывают каждый пиксель, TUI-приложения оперируют символами и управляют положением курсора в терминале. Архитектура такого приложения циклическая и событийно-ориентированная. Основной компонент – это бесконечный цикл обработки событий. На каждой итерации цикла программа выполняет три основных действия: ввод (чтение нажатий клавиш от пользователя), обработку (изменение внутреннего состояния модели в соответствии с командой) и вывод (перерисовку интерфейса на экране).

Особенностью TUI является работа с терминалом как с конечным автоматом. Терминал хранит у себя двумерную символьную матрицу (экран) и позицию курсора.

Архитектура управления выводом также требует отдельного внимания. Процесс отрисовки не должен выполняться после каждого малейшего изменения состояния, так как работа с терминалом – это операция ввода-вывода, которая может быть относительно медленной. Поэтому типичная TUI-архитектура использует механизм двойной буферизации вывода. Компонент представления анализирует текущее состояние модели и текущие размеры терминала (которые могут измениться, например, при изменении окна терминала), и формирует итоговую картинку, отправляя в терминал минимально необходимый набор команд для перемещения курсора и вывода символов. Особую роль играет обработка сигнала SIGWINCH, который генерируется при изменении размера окна терминала [3]. Получение этого сигнала должно вызывать пересчет геометрии интерфейса и полную перерисовку экрана, чтобы избежать некорректного отображения или обрезания текста.

Производительность TUI-приложения напрямую зависит от минимизации обращений к системным вызовам write. Каждый вывод символа может приводить к системному вызову, поэтому библиотеки используют буферизацию. Это классический алгоритм оптимизации вывода, который необходимо учитывать при разработке TUI-приложения.

1.2 Файловая система Unix

Прежде чем перейти к описанию внутренней модели данных, необходимо рассмотреть основные понятия и механизмы файловой системы Linux, поскольку именно они определяют требования к структурам,

хранящимся в памяти приложения [4]. Файловая система Linux представляет собой иерархическую структуру, организованную в виде единого дерева каталогов, корнем которого является каталог «/». Каждый элемент этой структуры – файл, каталог, символическая ссылка, устройство и т.д. – описывается индексным дескриптором (inode), который хранит метаданные (владелец, права доступа, временные метки, размер, расположение блоков данных на диске), но не содержит имени файла. Имена связываются через записи в каталогах. Такая организация позволяет иметь несколько жёстких ссылок на один и тот же индексный дескриптор, что стоит учитывать при навигации [5].

Для взаимодействия с файловой системой из пользовательского пространства Linux предоставляет ряд системных вызовов, являющихся основой для любых операций [6]. Навигатор будет активно использовать следующие:

– opendir, readdir, closedir – для чтения содержимого каталогов. Эти вызовы возвращают записи каталога, каждая из которых содержит имя файла и номер inode;

– stat, lstat, fstat – для получения метаданных файла. lstat отличается тем, что в случае символической ссылки возвращает информацию о самой ссылке, а не о файле, на который она указывает, что критически важно для корректного построения дерева без раскрытия ссылок;

– open, close, read, write – для работы с содержимым файлов (например, при копировании);

– mkdir, rmdir, unlink, rename – для создания и удаления каталогов и файлов, а также перемещения;

– chmod, chown – для изменения прав доступа и владельца (могут потребоваться при копировании с сохранением атрибутов).

Обычно, системные вызовы не вызываются напрямую, это делается через обёрточную функцию из glibc (или другой библиотеки). Часто, но не всегда, имя обёрточной функции совпадает с именем системного вызова, который она вызывает. Например, в glibc есть функция chdir(), которая вызывает делающий всё работу системный вызов «chdir».

Часто, обёрточная функция glibc очень маленькая, она просто копирует аргументы в нужные регистры перед запуском системного вызова, а затем присваивает переменной errno значение, которое было возвращено системным вызовом.

Иногда, однако, обёрточная функция производит дополнительную работу до осуществления системного вызова. Например, в настоящее время существует два похожих системных вызова – truncate(2) и truncate64(2).

Основные уровни файловой иерархии:

– */bin* – основные двоичные файлы;

– */etc* – конфигурационные файлы;

– */home* – пользовательские данные;

– */var* – лог-файлы, динамические данные;

– */boot* – загрузочные файлы (*vmlinuz, initrd*);

– */usr* – второстепенные программы и библиотеки.

Linux поддерживает широкий спектр файловых систем: *ext4*, *XFS*, *Btrfs*, *ZFS* и другие. Современные дистрибутивы используют systemd-tmpfiles или такие технологии как *tmpfs* для управления временными директориями [7].

Использование общепринятых имен основных файлов и структуры каталогов существенно облегчает работу в операционной системе, её администрирование и переносимость. Некоторые из этих структур используются при запуске системы, некоторые – во время работы, но все они имеют большое значение для ОС в целом, а нарушение этой структуры может привести к неработоспособности системы или ее отдельных компонентов. Структура файловой системы проиллюстрирована на рисунке 1.1.

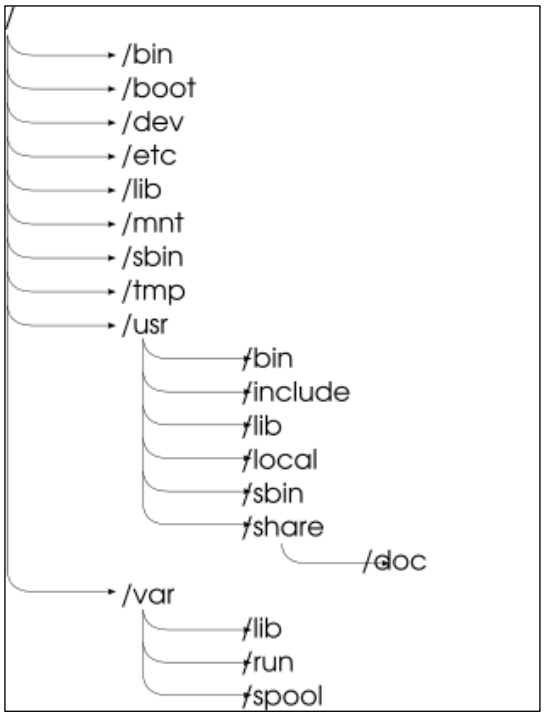


Рисунок 1.1 – Стандартные каталоги файловой системы в *Unix*

1.3 Модель представления файловой системы в памяти

Модель представления файловой системы в памяти определяет способ хранения и организации данных о файлах и каталогах, необходимых для работы консольного навигатора, и её выбор напрямую влияет на производительность, потребление ресурсов и функциональные возможности приложения. Основное назначение модели заключается в обеспечении быстрого доступа к информации о содержимом каталогов, а также к метаданным файлов, таким как права доступа, размер и время изменения, для последующего отображения пользователю и выполнения операций копирования, перемещения или удаления. Модель должна позволять эффективно перемещаться по иерархической структуре файловой системы и оперативно реагировать на изменения, происходящие во внешней среде.

Файловая система имеет древовидную структуру, и в памяти это дерево может храниться различными способами. Первый подход предполагает построение полного дерева, где каждый узел соответствует файлу или каталогу и содержит имя, путь, метаданные и ссылки на дочерние элементы. Такое решение обеспечивает быстрый доступ к любой части дерева и удобно для отображения иерархии, однако оно требует значительного расхода памяти при работе с глубокими или обширными файловыми системами, а также нуждается в постоянной синхронизации с реальными изменениями на диске. Второй подход основан на ленивой, или динамической, загрузке, при которой в памяти хранится только информация о текущем каталоге и, возможно, о родительских путях, а при переходе в подкаталог его содержимое загружается с диска. Такой метод экономит память и гарантирует актуальность данных, но увеличивает количество обращений к файловой системе, что характерно для многих навигаторов.

Независимо от выбранного подхода, модель обычно включает несколько ключевых компонентов. Узел представляет собой файл или каталог и содержит имя, тип, метаданные, путь, а для каталога также список дочерних узлов в случае полного дерева. Текущий контекст хранит информацию о положении пользователя в иерархии, например указатель на текущий каталог, список выделенных элементов и режим отображения в виде плоского списка или дерева. При использовании ленивой загрузки может применяться кэш метаданных для ограниченного числа элементов, чтобы ускорить повторный доступ к часто используемым данным, например к родительским каталогам.

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Библиотеки управления терминалом

Терминал Linux может находиться в различных режимах, управлять положением курсора, цветом, атрибутами символов и обрабатывать специальные клавиши. Для создания интерактивных консольных приложений, таких как файловый навигатор, необходимо низкоуровневое или высокоуровневое управление терминалом. В Linux основными средствами такого управления являются системный интерфейс termios и библиотека ncurses.

Прежде чем рассматривать библиотеки, важно понять, что терминал в Linux по умолчанию работает в каноническом режиме [8]. В этом режиме ввод буферизуется построчно, доступна базовая редактирование строки (backspace, удаление символов), а передача данных программе происходит только после нажатия Enter. Такой режим удобен для командной строки, но совершенно не подходит для интерактивных приложений, где требуется немедленная реакция на каждое нажатие клавиши (стрелки, функциональные клавиши, одиночные символы). Кроме того, в каноническом режиме эхо введённых символов обычно включено, что также мешает построению собственного интерфейса.

Понимая компоненты архитектуры Linux, пользователи могут лучше настраивать, устранять неполадки в системе, в конечном итоге делая Linux идеальным выбором для широкого спектра вычислительных задач.

Для переключения терминала в режим, пригодный для TUI, используются возможности termios – интерфейса терминального ввода-вывода, определённого в POSIX.

Termios представляет собой набор структур и функций, позволяющих изменять параметры терминального устройства. Основная структура – struct termios, которая содержит флаги, управляющие различными аспектами работы терминала: режим ввода (c_iflag), режим вывода (c_oflag), управляющие флаги (c_cflag), локальные флаги (c_lflag) и массив управляющих символов (c_cc).

Все изменения выполняются с помощью функций tcgetattr() (получить текущие параметры) и tcsetattr() (установить новые). Прямая работа с termios даёт полный контроль над терминалом, но требует от разработчика самостоятельной обработки всех аспектов: чтение многобайтовых последовательностей функциональных клавиш, управление курсором, цветом и т.д.

Библиотека ncurses является стандартом для создания текстовых интерфейсов в Unix-подобных системах. Она предоставляет удобный программный интерфейс, скрывающий детали работы с терминалом и управляющими последовательностями. Ncurses берёт на себя следующие задачи:

1 Инициализацию и восстановление терминала.

2 Переключение в нужный режим ввода (аналогично termios, но автоматически).

3 Чтение клавиш с распознаванием функциональных клавиш (стрелки, F1–F12, Home, End и др.) – библиотека возвращает специальные константы, такие как KEY_UP, KEY_DOWN.

4 Вывод текста с автоматическим переносом, форматированием, управлением позицией курсора.

5 Поддержка окон и подокон: можно создавать перекрывающиеся области экрана, каждую со своим курсором и буфером.

6 Цветовое оформление: определение пар «цвет текста / цвет фона» и их использование при выводе.

7 Работа с мышью.

8 Оптимизация вывода: ncurses обновляет только изменившиеся части экрана, минимизируя объём передаваемых данных.

Типичная структура программы с ncurses включает вызов initscr() для инициализации, настройку параметров (cbreak(), noecho(), keypad()), затем основной цикл с обработкой клавиш (getch()) и обновлением экрана (refresh()). По окончании вызывается endwin() для восстановления терминала.

Недостатком ncurses можно считать некоторое увеличение размера программы и необходимость изучения API, но для серьёзных проектов эти затраты окупаются.

2.2 Системные API файловой системы

Для выполнения любых операций с файловой системой в Linux приложение обязано взаимодействовать с ядром через системные вызовы. Системные вызовы представляют собой способ перехода из пользовательского пространства в пространство ядра для выполнения привилегированных действий, таких как создание, удаление, чтение файлов или получение их атрибутов. В консольном навигаторе все операции – навигация, просмотр, копирование, перемещение, удаление, создание – в конечном счёте реализуются через соответствующие системные вызовы или библиотечные обёртки над ними.

Непосредственная работа с системными вызовами в коде на C/C++ обычно осуществляется через функции-обёртки, предоставляемые стандартной библиотекой языка (glibc). Эти обёртки скрывают детали передачи управления ядру и обеспечивают совместимость на уровне исходного кода.

Для навигации и определения текущего состояния используются вызовы getcwd для получения абсолютного пути текущего рабочего каталога и chdir для его смены. Чтение содержимого каталогов выполняется с помощью вызовов opendir, readdir и closedir. Функция readdir возвращает структуру dirent, содержащую имя файла и его номер индексного дескриптора, что достаточно для отображения списка, но не для получения полной информации об объекте.

Для получения метаданных (тип файла, размер, права доступа, время модификации) используются вызовы stat, lstat или fstat. Вызов lstat, в отличие

от stat, он возвращает информацию о самой ссылке, а не о файле, на который она указывает. Это позволяет отображать ссылку как отдельный объект и избежать нежелательного перехода по ней или попыток рекурсивного обхода.

Операции модификации файловой системы реализуются через вызовы rename для перемещения и переименования, unlink для удаления файлов, rmdir для удаления пустых каталогов, mkdir для создания новых каталогов. Для копирования файлов используются open, read и write в сочетании: исходный файл открывается на чтение, целевой – на запись с необходимыми правами, после чего данные считываются во временный буфер и записываются в целевой файл. При копировании важно также использовать вызовы fstat и fchmod для сохранения прав доступа исходного файла, если это требуется. Изменение прав доступа выполняется вызовом chmod, смена владельца – chown.

Все вышеперечисленные вызовы могут завершаться ошибкой. Для диагностики и вывода сообщений пользователю программа должна анализировать errno и сопоставлять его с конкретной ситуацией. Типичные ошибки включают EACCES (отсутствие прав доступа), ENOENT (файл или каталог не найден), ENOTEMPTY (каталог не пуст), EBUSY (ресурс занят) и другие.

При разработке навигатора для Unix-подобных систем также необходимо учитывать особенности работы с путями. Системные вызовы ожидают абсолютные или относительные пути в кодировке UTF-8. При работе с символическими ссылками и жёсткими ссылками следует помнить, что один и тот же индексный дескриптор может иметь несколько имён, что может привести к неоднозначности при навигации или подсчёте размера. Дополнительно в Linux существуют специфичные вызовы, такие как readlink для чтения цели символической ссылки, и realpath для канонизации пути (устранения символических ссылок и точек). Их использование может повысить точность отображения состояния файловой системы.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

3.1 Обоснование необходимости разработки

С момента появления операционных систем, работающих с файлами на дисках, возникла потребность в инструментах для управления этими файлами. Первые файловые менеджеры представляли собой простые командные интерпретаторы, где пользователь вынужден был вводить команды типа cp, mv, rm и помнить структуру каталогов. С развитием вычислительной техники и появлением графических интерфейсов стали популярны файловые менеджеры с визуальным представлением, такие как Norton Commander в среде DOS [9], интерфейс представлен на рисунке 3.1. Однако консольные файловые менеджеры не утратили своей актуальности. Они остаются востребованными среди системных администраторов, разработчиков встраиваемых систем, а также пользователей, работающих в средах без графической оболочки, например на серверах или в минималистичных дистрибутивах Linux. Консольные навигаторы позволяют эффективно перемещаться по файловой системе, выполнять операции с файлами, используя минимальные ресурсы, что важно при удалённом доступе по SSH, где графический интерфейс либо отсутствует, либо работает медленно.

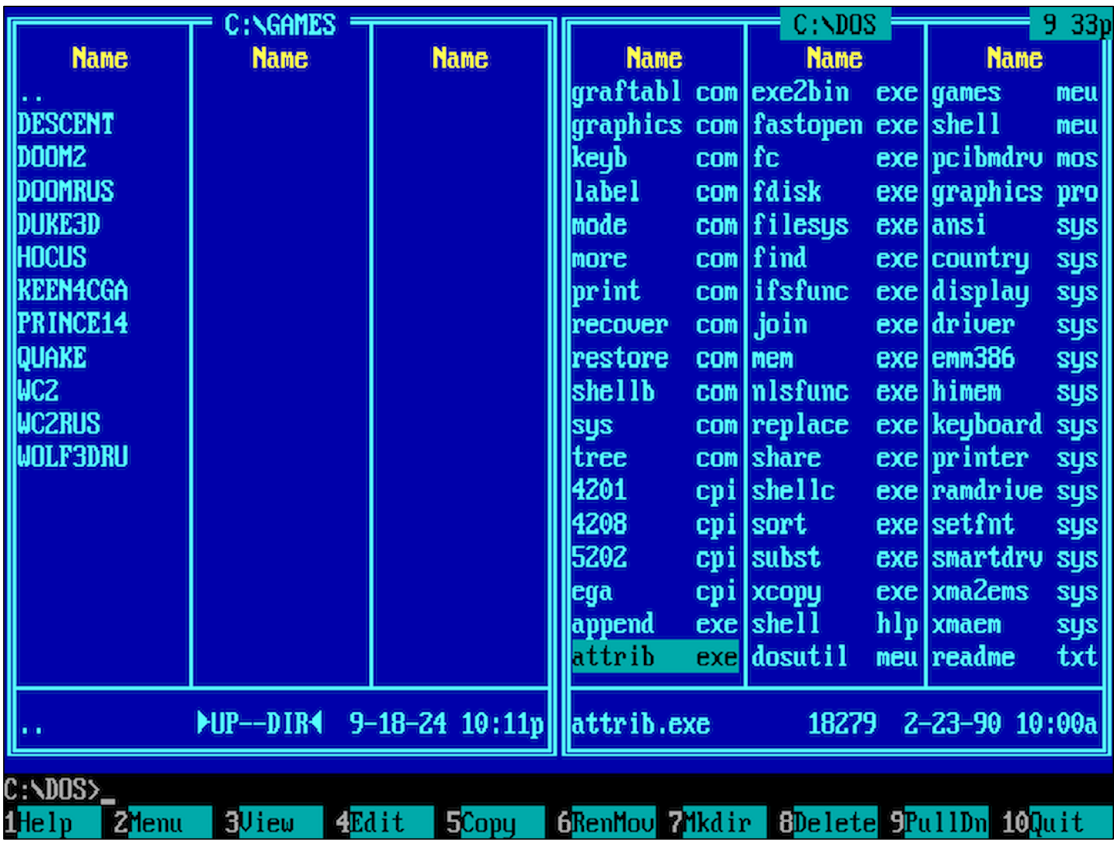


Рисунок 3.1 – Norton Commander в среде DOS

Разработка консольного навигатора преследует несколько целей. Во-первых, это глубокое погружение в системные вызовы и механизмы взаимодействия прикладного кода с ядром операционной системы. Практический опыт работы с такими вызовами, как opendir, readdir, stat, chdir, open, read, write, unlink, mkdir, rename, а также учится корректно обрабатывать ошибки, анализируя переменную errno. Во-вторых, создание текстового интерфейса пользователя (TUI) требует понимания терминальных возможностей, работы с библиотекой ncurses или прямым управлением termios, что расширяет знания в области низкоуровневого ввода-вывода. В-третьих, реализация навигатора заставляет детально изучить архитектуру файловой системы, права доступа, модель безопасности, что является основой для дальнейшей работы с UNIX-подобными системами.

Кроме образовательных задач, разрабатываемый продукт может служить основой для более сложных утилит, например резервного копирования, синхронизации данных, аудита файловой системы. Консольный навигатор с открытым исходным кодом также может быть полезен сообществу как пример учебного проекта, демонстрирующего грамотное использование системных вызовов и обработку нештатных ситуаций.

В настоящее время существует множество консольных файловых менеджеров, каждый из которых предлагает свой набор возможностей, но ни один из них не является универсальным решением, полностью удовлетворяющим всем требованиям. Классическим представителем является Midnight Commander (MC), который на протяжении десятилетий остаётся стандартом де-факто для системных администраторов. Интерфейс Midnight Commander представлен на рисунке 3.2.

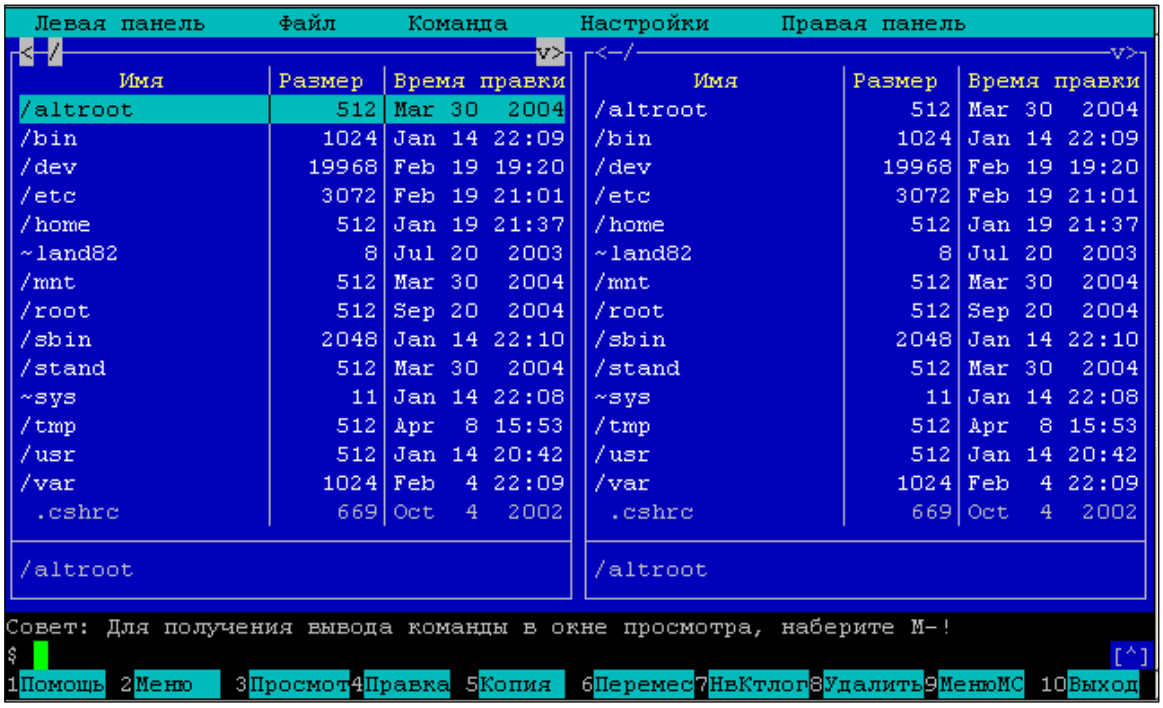


Рисунок 3.2 – Midnight Commander

MC обладает развитым двухпанельным интерфейсом, встроенным редактором, поддержкой FTP и SCP, а также широкими возможностями настройки. Однако его кодовая база насчитывает десятки тысяч строк на C и зависит от библиотеки ncurses, что делает проект сложным для изучения и модификации.

Более современные альтернативы, такие как ranger, предлагают визуально привлекательный интерфейс с поддержкой вкладок, предварительным просмотром файлов и управлением с помощью сочетаний клавиш. Но ranger написан на Python, что накладывает дополнительные требования к окружению и может снижать производительность на маломощных системах или при работе с большим количеством файлов. Его интерфейс можно увидеть на рисунке 3.3.

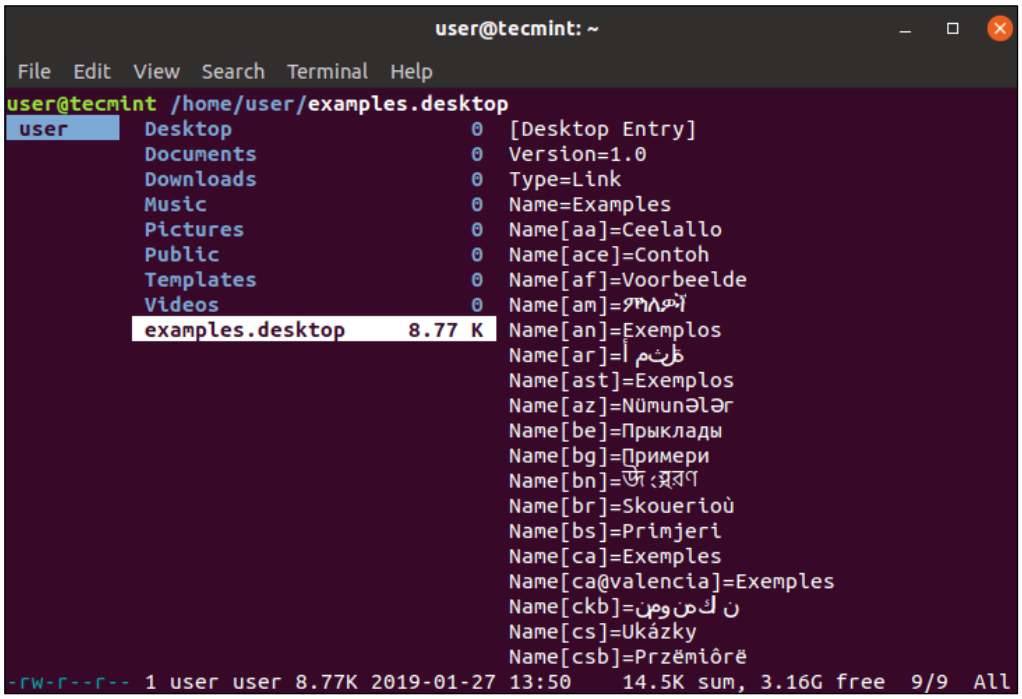


Рисунок 3.3 – Ranger на Linux Ubuntu

Другой подход реализован в nnn – быстром и минималистичном менеджере, который фокусируется на эффективности использования ресурсов и скорости работы. Однако управление требует запоминания множества горячих клавиш, что создаёт порог вхождения для новых пользователей. Интерфейс данного навигатора представлен на рисунке 3.4.

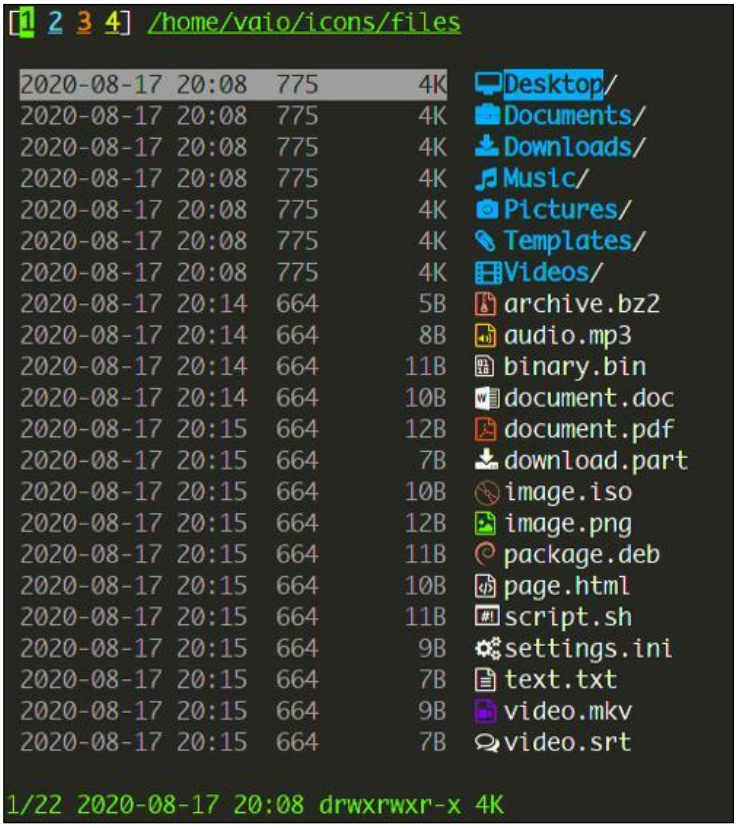


Рисунок 3.4 – nnn с иконками в Linux

Утилита lf, написанная на Go, сочетает простоту конфигурации с высокой производительностью, но её функциональность во многом определяется внешними скриптами, что усложняет развёртывание в изолированных средах. Интерфейс представлен на рисунке 3.5.

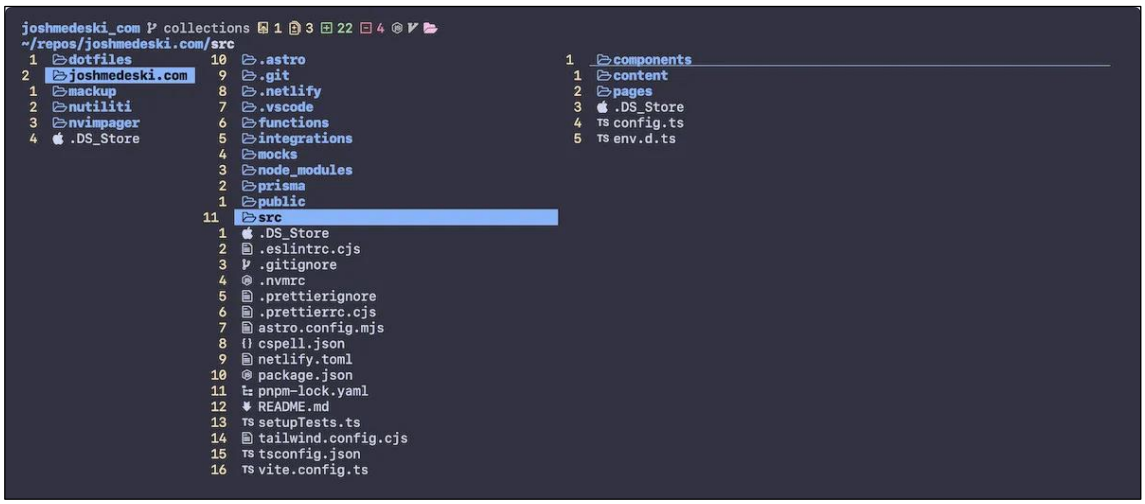


Рисунок 3.5 – Визуальный интерфейс lf

Графические файловые менеджеры (Nautilus, Dolphin, Thunar и другие) предлагают интуитивно понятный интерфейс, но они требуют наличия графической подсистемы и не могут использоваться в сеансах удалённого

доступа по SSH или на серверах без графической оболочки. Кроме того, их архитектура часто завязана на конкретные оконные окружения (GNOME, KDE), что делает их перенос на другие платформы затруднительным. Графический файловый менеджер Nautilus представлен на рисунке 3.6, остальные рассматривать не будем, так как нашей основной целью является консольный файловый менеджер.

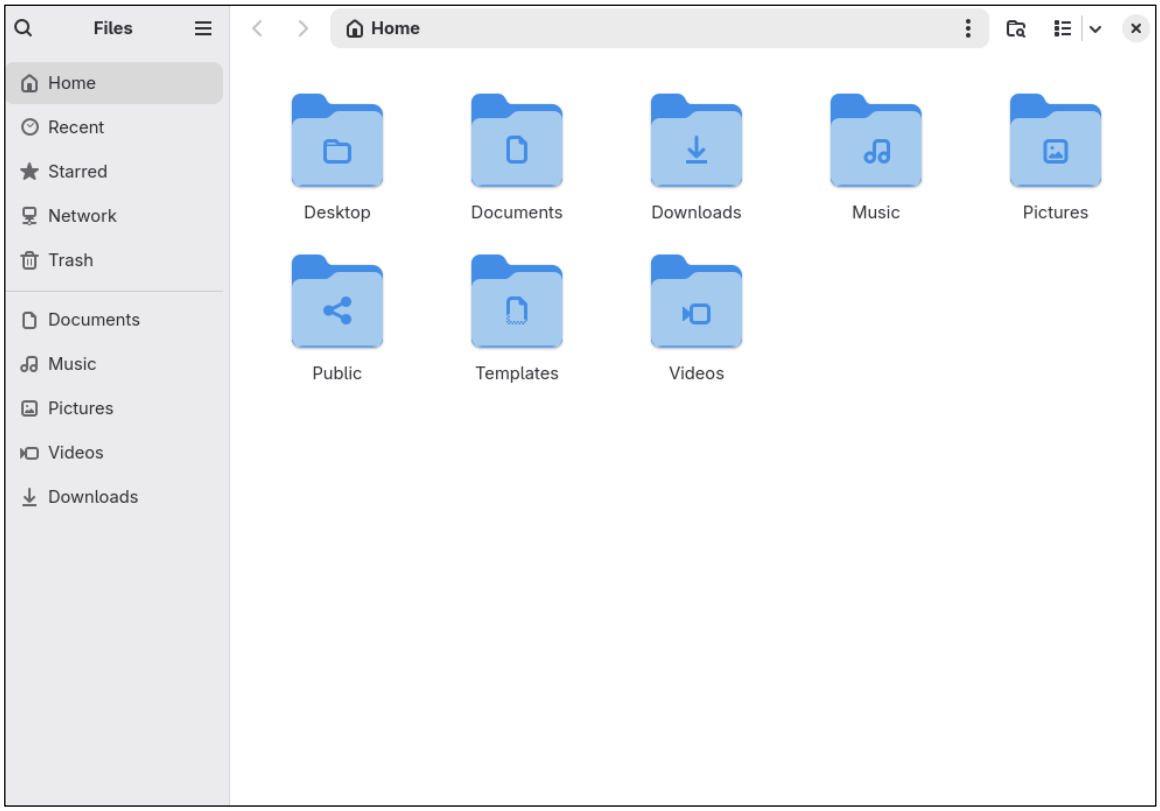


Рисунок 3.6 – Визуальный интерфейс Nautilus

Графический интерфейс графического файлового менеджера Dolphin представлен на рисунке 3.7.

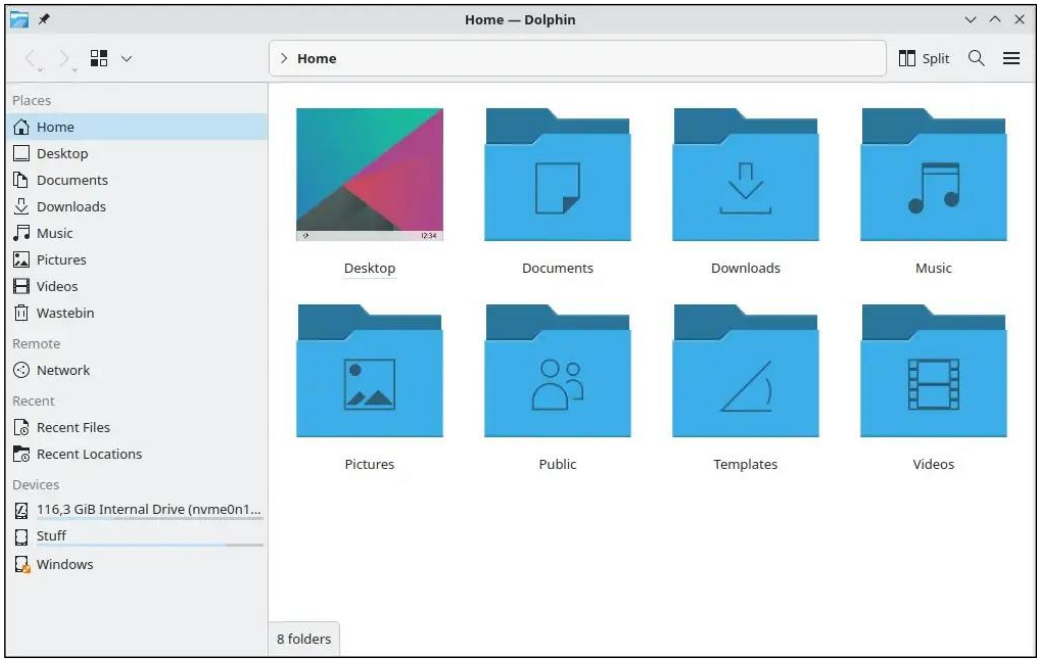


Рисунок 3.7 – Визуальный интерфейс Dolphin

Графический интерфейс графического файлового менеджера Dolphin представлен на рисунке 3.8.

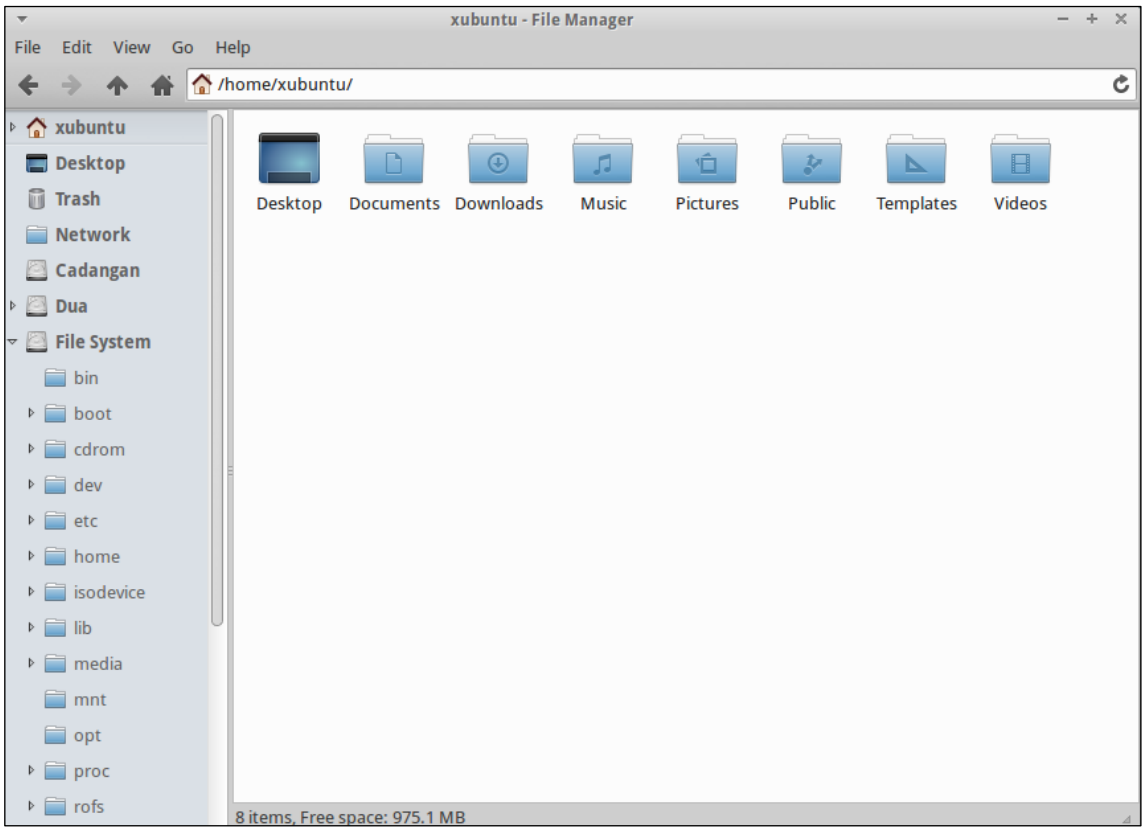


Рисунок 3.8 – Визуальный интерфейс Thunar

Таким образом, каждое из существующих решений имеет свои ограничения: либо избыточная сложность и зависимость от внешних библиотек, либо недостаточная гибкость для образовательных целей, либо отсутствие встроенных средств тестирования и документирования низкоуровневых механизмов работы с файловой системой. Разрабатываемый консольный навигатор восполняет этот пробел, поскольку он:

1 Написан на C++ с использованием только стандартных библиотек и системных вызовов POSIX, что делает его максимально переносимым и не требующим установки дополнительных пакетов.

2 Обладает прозрачной модульной архитектурой, где каждая операция (копирование, перемещение, удаление) реализована в отдельной функции, что позволяет легко анализировать и модифицировать код.

3 Включает встроенные тесты корректности и производительности, которые могут служить эталоном для проверки работы файловых операций в учебных целях.

4 Демонстрирует практическое применение низкоуровневого ввода-вывода (управление терминалом через termios, работа с каталогами через opendir/readdir, получение метаданных через stat, рекурсивный обход файловой системы).

5 Может быть использован как основа для более сложных проектов (резервное копирование, аудит файлов, синхронизация данных) благодаря своей простоте и отсутствию избыточных зависимостей.

Следовательно, разработка нового консольного навигатора оправдана не только потребностью в компактном и эффективном инструменте для работы в консольной среде, но и уникальной возможностью систематизировать знания о взаимодействии прикладного кода с операционной системой, создав при этом полнофункциональный продукт, пригодный для повседневного использования и дальнейшего развития.

Таким образом, необходимость разработки обусловлена сочетанием следующих факторов: потребностью в удобном инструменте для работы в консольной среде, ограничениями существующих решений, а также образовательной ценностью проекта, позволяющего изучить принципы функционирования операционных систем и программной архитектуры.

3.2 Иерархическая структура файловых систем

Файловые системы современных операционных систем, в частности Unix-подобных, организованы в виде иерархической древовидной структуры. Такая модель представляет собой упорядоченное множество объектов, где каждый элемент (за исключением корневого) имеет единственного родителя, что обеспечивает логичную и понятную организацию данных.

Корнем иерархии является каталог «/», от которого отходят все остальные каталоги и файлы. Каждый каталог может содержать как файлы, так и подкаталоги, формируя тем самым вложенную структуру произвольной глубины. Такой подход обеспечивает удобство группировки данных и их

логическую изоляцию.

С теоретической точки зрения файловая система может рассматриваться как ориентированное дерево, где вершины соответствуют объектам файловой системы, а рёбра – отношениям «родитель–потомок». Это представление позволяет применять классические алгоритмы обхода деревьев (например, поиск в глубину или ширину) при реализации навигации и обработки данных.

Особенностью Unix-подобных систем является разделение имени файла и его содержимого. Каждый объект описывается индексным дескриптором (inode), который содержит метаданные и указатели на блоки данных. Имена файлов хранятся в каталогах как ссылки на соответствующие inode. Это позволяет реализовать механизм жёстких ссылок, при котором один и тот же файл может иметь несколько имён в разных каталогах.

Дополнительно в иерархии присутствуют символические ссылки, которые представляют собой специальные файлы, содержащие путь к другому объекту. В отличие от жёстких ссылок, они могут указывать на файлы в других файловых системах и могут быть «разорванными», если целевой объект удалён.

Иерархическая структура обеспечивает ряд преимуществ:

– логическую организованность данных;

– масштабируемость при увеличении объёма информации;

– удобство реализации прав доступа и разграничения ресурсов;

– универсальность для различных типов устройств и файлов.

Однако такая структура накладывает и определённые требования на программные средства, работающие с ней. В частности, необходимо учитывать возможность циклов при обработке символических ссылок, корректно обрабатывать права доступа, а также эффективно выполнять операции обхода дерева без избыточного потребления ресурсов и нагрузки на операционную систему.

В контексте разработки файлового навигатора иерархическая модель определяет внутреннее представление данных в памяти и алгоритмы навигации. Программа должна обеспечивать быстрый переход между уровнями дерева, отображение содержимого каталогов и корректную работу с различными типами файловых объектов.

3.3 Права доступа в Unix/Windows

Механизм разграничения доступа является одной из ключевых составляющих любой операционной системы и играет важную роль при разработке файлового навигатора. Он определяет, какие действия пользователь или процесс может выполнять над файлами и каталогами, обеспечивая безопасность и целостность данных.

В Unix-подобных системах используется модель прав доступа, основанная на трёх категориях пользователей: владелец (user), группа (group) и остальные (others). Для каждой категории задаются три типа разрешений: чтение (r), запись (w) и выполнение (x) [10]. Эти права представлены в виде

битовой маски и могут быть отображены, например, как «rwxr-xr--», что продемонстрированно на рисунке 3.9.

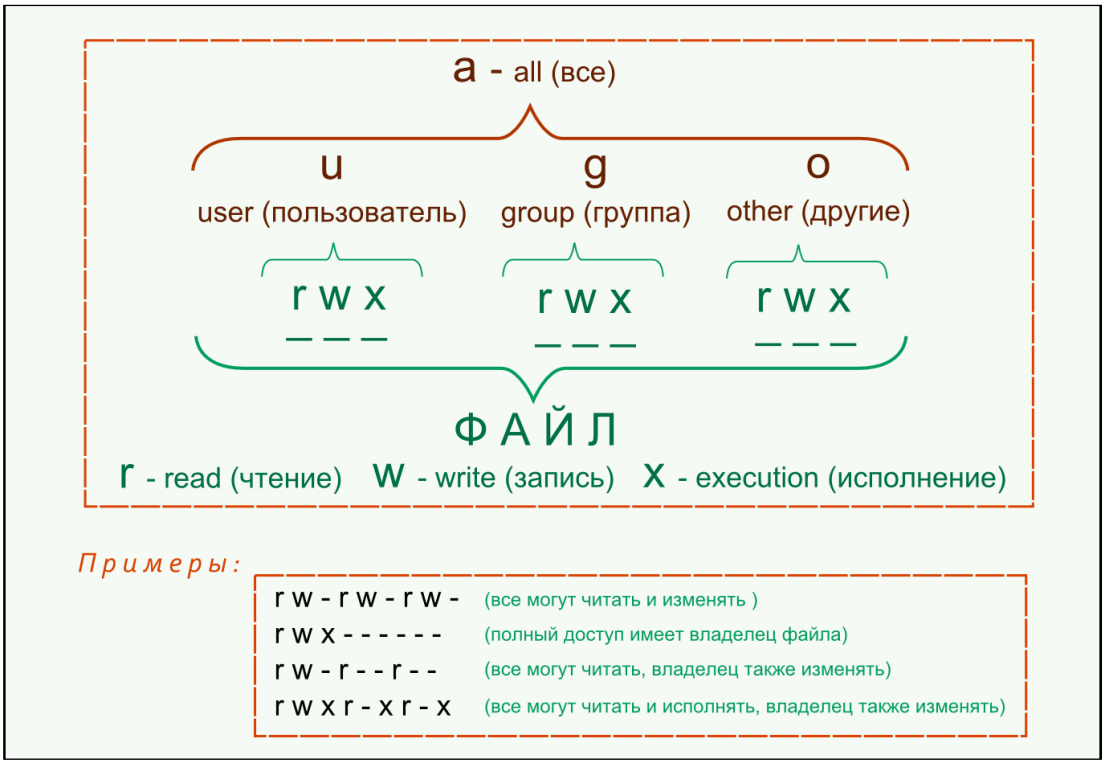


Рисунок 3.9 – Права доступа в Unix-подобных системах

Чтение позволяет просматривать содержимое файла или список файлов в каталоге, запись – изменять содержимое или структуру каталога, выполнение – запускать файл как программу или входить в каталог. Для каталогов интерпретация прав несколько отличается: право выполнения означает возможность перехода в каталог, а право чтения – возможность просмотра его содержимого.

Дополнительно в Unix существуют специальные биты:

– setuid – выполнение файла с правами владельца;

– setgid – выполнение с правами группы;

– sticky bit – ограничение удаления файлов в каталоге (например, в /tmp).

В операционной системе Windows используется более сложная модель, основанная на списках управления доступом (ACL). В этой модели каждому объекту сопоставляется список записей, определяющих права для конкретных пользователей или групп [11]. Права могут быть более детализированными (например, отдельные разрешения на чтение атрибутов, изменение прав, удаление и т.д.). Визуальное представление прав доступа в Windows представлено на рисунке 3.10.

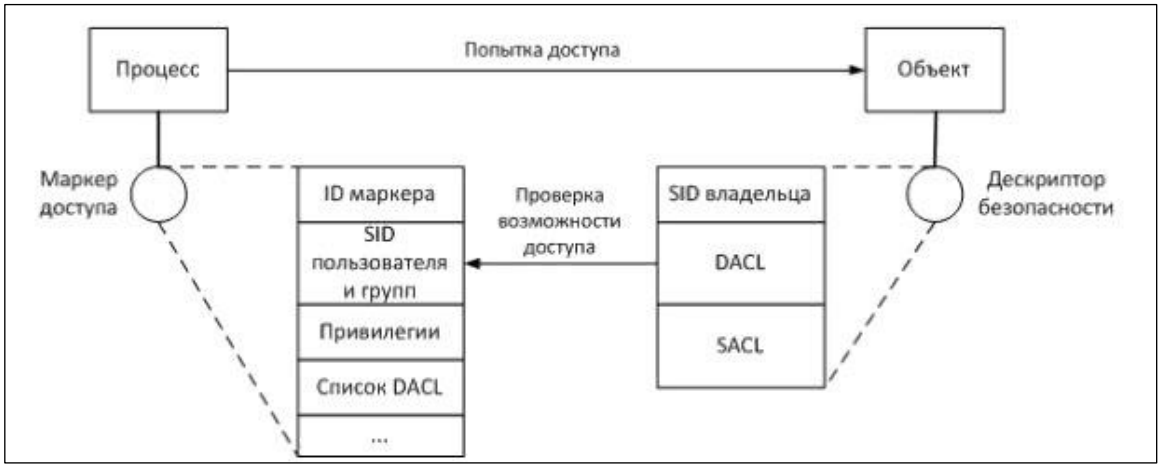


Рисунок 3.10 – Права доступа в системе Windows

Основное отличие заключается в том, что Unix использует компактную и простую модель, удобную для быстрого анализа и реализации, тогда как Windows предоставляет более гибкую, но сложную систему управления доступом.

Для кроссплатформенных приложений это различие представляет определённую сложность, однако в рамках разработки консольного навигатора для Linux приоритет отдаётся Unix-модели. Программа должна корректно считывать права доступа с помощью системных вызовов и отображать их пользователю в привычном формате.

Также важно учитывать обработку ошибок, связанных с доступом. Например, попытка открыть файл без соответствующих прав должна корректно обрабатываться и сопровождаться информативным сообщением. Это повышает надёжность и удобство использования приложения.

Таким образом, механизм прав доступа является неотъемлемой частью теоретического обоснования разработки, поскольку он напрямую влияет на функциональность навигатора, его безопасность и корректность работы в различных пользовательских сценариях.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

4.1 Описание обработки ввода пользователя

Обработка пользовательского ввода является основой интерактивного взаимодействия с программой. Для обеспечения мгновенной реакции на нажатия клавиш без необходимости подтверждения клавишей Enter используется функция getch, которая переводит терминал в неканонический режим. В этом режиме отключаются буферизация строки и эхо-вывод нажатых символов, что позволяет считывать символы по одному. После получения символа настройки терминала восстанавливаются, что гарантирует корректную работу других приложений после завершения программы.

Запуск программы начинается с запроса пути, как показано на рисунке 4.1. Пользователю предлагается ввести путь к каталогу, который станет начальным, или нажать Enter для использования текущего рабочего каталога. Этот этап реализован с помощью std::getline и проверки существования каталога через функцию is_directory, которая использует системный вызов stat. Программа не завершается при ошибочном вводе, а продолжает запрашивать путь, пока не будет получен корректный путь.

```
[koxan@victusMonjaro          ]$ ./file_manager
Enter path, or press Enter to use current directory [/home/koxan/VScode/osisp_CW]:
```

Рисунок 4.1 – Запуск программы

После установки начального каталога программа входит в бесконечный цикл, в котором последовательно отображается текущее состояние интерфейса (список файлов или древовидное представление) и вызывается функция handle_input. Внутри handle_input каждый раз заново считывается содержимое текущей директории с помощью opendir и readdir. Имена сортируются в алфавитном порядке с помощью std::sort. Такой подход гарантирует, что навигация всегда оперирует актуальным списком объектов, даже если файловая система изменилась в результате операций копирования, удаления или создания.

Навигация по списку осуществляется клавишами w (вверх) и s (вниз). Индекс выбранного элемента selected_index изменяется в пределах от 0 до entries.size() – 1. Весь визуальный интерфейс текущего каталога представлен на рисунке 4.2.

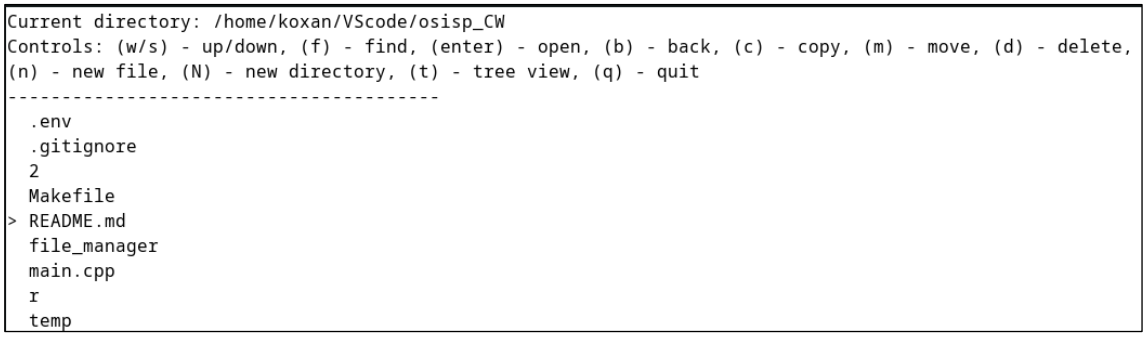


Рисунок 4.2 – Визуальный интерфейс

Для быстрого поиска объекта по имени предусмотрена клавиша f. При её нажатии программа выводит приглашение Enter name to find: и считывает строку. Затем с помощью алгоритма std::find проверяется наличие введённого имени в векторе entries. Если элемент найден, selected_index устанавливается на его позицию, что перемещает курсор к найденному объекту. В противном случае выводится сообщение not found. Press any key to continue, как это показано на рисунке 4.3, где пользователь пытался найти несуществующий файл. Такая реализация поиска не требует дополнительного переключения режимов и интуитивно понятна.

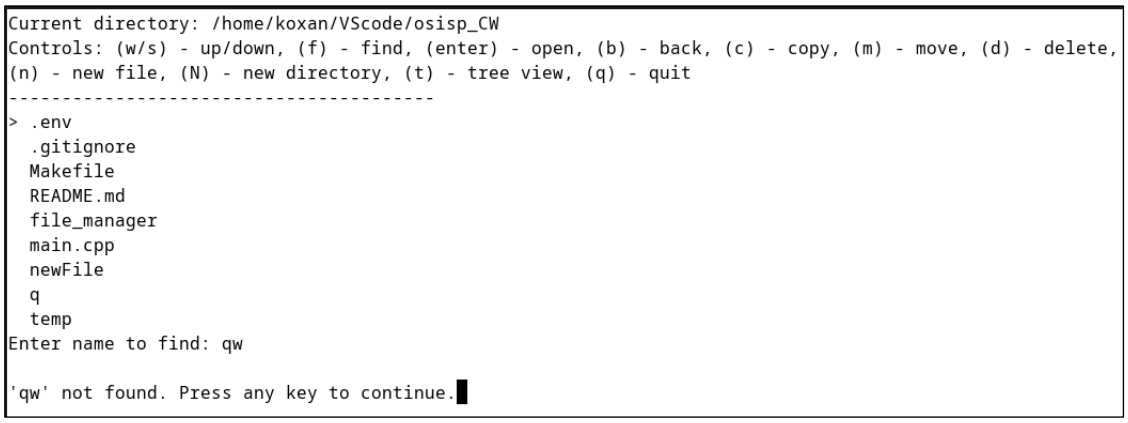


Рисунок 4.3 – Быстрый поиск элемента

Клавиша Enter обрабатывает выбор текущего объекта. Если выбранный элемент является каталогом, программа обновляет current_path, добавляя имя подкаталога, и сбрасывает selected_index в ноль, обеспечивая переход внутрь каталога. Если выбран файл, вызывается функция view_file, которая отображает его метаданные и содержимое. Переход на уровень выше осуществляется клавишей b. Функция get_parent_path выделяет родительский каталог, отсекая последний сегмент пути. Для корневого каталога переход не выполняется, а для путей вида /home/user возвращается /. После смены каталога индекс выделения сбрасывается, чтобы курсор оказался в начале нового списка.

Для выполнения файловых операций используются клавиши: c

(копирование), m (перемещение), d (удаление), n (создание файла), N (создание каталога). Перед выполнением каждой из этих операций программа запрашивает дополнительную информацию у пользователя. Например, при копировании или перемещении выводится запрос Enter destination path:, как видно на рисунке 4.4. Введённый путь может быть как абсолютным, так и относительным.

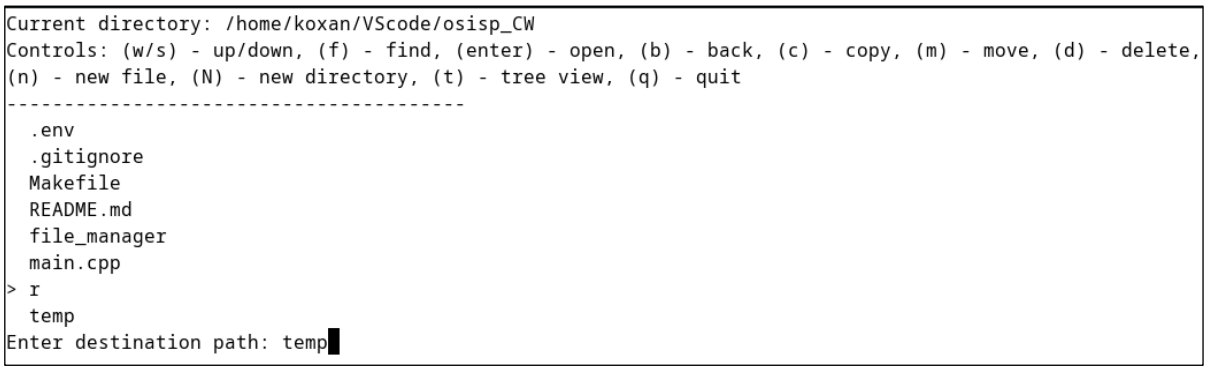


Рисунок 4.4 – Запрос конечного пути

Функции copy_item и move_item преобразуют относительные пути, добавляя к ним текущий каталог. При удалении перед выполнением команды выводится запрос подтверждения Are you sure you want to delete? (y/n): (демонстрация этого на рисунке 4.5). Это предотвращает случайное удаление важных данных.

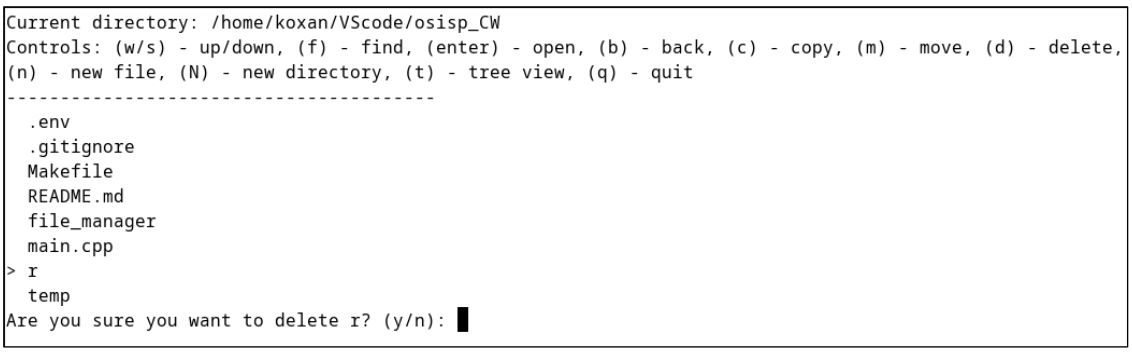


Рисунок 4.5 – Подтверждение операции удаления

Создание новых объектов также сопровождается вводом имени: для файла используется запрос Enter new file name:, для каталога – Enter new directory name: (это видно на рисунке 4.6).

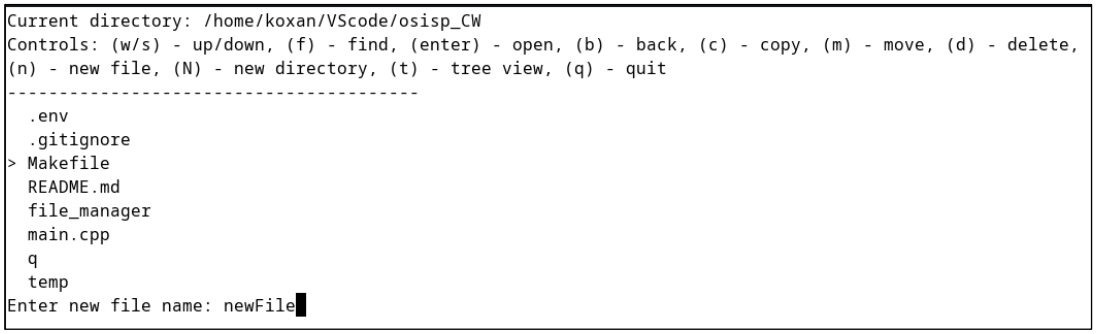


Рисунок 4.6 – Создание новых объектов

После создания нового файла пользователь может сразу открыть его для просмотра, что демонстрирует рисунок 4.7, где отображаются метаданные пустого файла newFile.

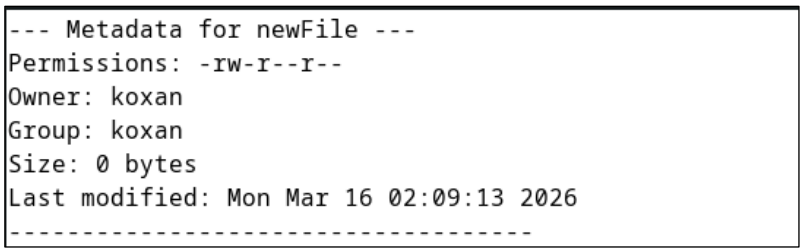


Рисунок 4.7 – Метаданные файла

Клавиша t переключает режим отображения между стандартным списком и древовидным представлением. Глобальная переменная tree_view_enabled хранит текущее состояние, и в зависимости от её значения вызывается либо display_directory, либо display_tree. Вывод в виде списка был представлен выше, а в виде дерева можно увидеть на рисунке 4.8.

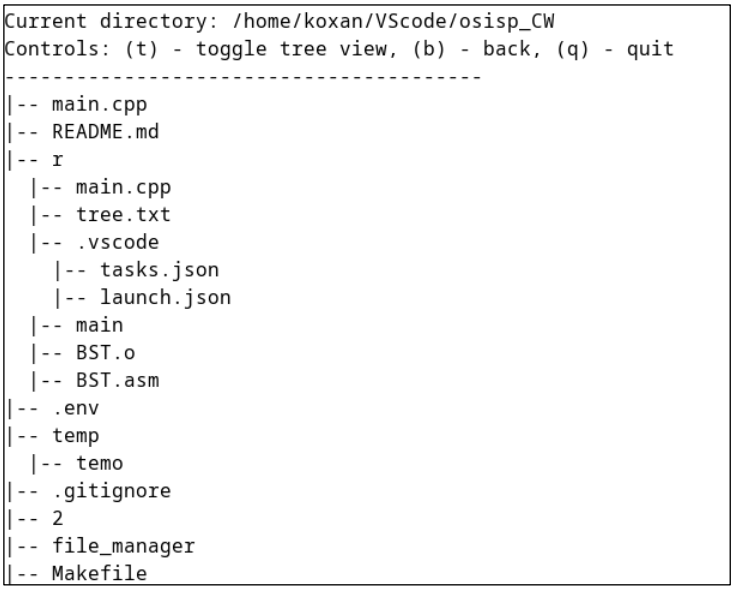


Рисунок 4.8 – Отображение директорий в виде дерева

Завершение работы программы происходит по нажатию q, что приводит к вызову exit(0).

4.2 Описание логики отрисовки дерева

Древовидное представление файловой системы активируется нажатием клавиши t и позволяет пользователю визуально оценить иерархическую структуру каталогов без необходимости последовательного перехода между ними. За отрисовку дерева отвечает рекурсивная функция display_tree, которая принимает два параметра: путь к корневому каталогу и текущий уровень вложенности.

При вызове display_tree открывается директория с помощью opendir. Если открытие не удаётся, выводится сообщение об ошибке с помощью perror, и функция завершается. В цикле обрабатываются все записи каталога. Записи с именами . и .. игнорируются, чтобы избежать зацикливания и загромождения вывода. Для каждой оставшейся записи сначала формируется отступ: цикл for (int i = 0; i < level; ++i) std::cout << " "; выводит по два пробела на каждый уровень вложенности, создавая визуальный сдвиг. Затем выводится строка |-- и имя элемента. После этого строится полный путь к элементу через конкатенацию path + "/" + entry->d_name, и с помощью функции is_directory проверяется, является ли элемент каталогом. Если да, то функция display_tree рекурсивно вызывается для этого подкаталога с увеличенным на единицу уровнем.

Рекурсивный обход позволяет построить дерево любой глубины, ограниченной только возможностями файловой системы. В верхней части экрана отображается текущий каталог и краткая подсказка по управлению: (t) - toggle tree view, (b) - back, (q) - quit. Далее следует древовидный вывод, где каждый элемент начинается с |--, а вложенные элементы имеют дополнительные отступы. Такое представление наглядно демонстрирует структуру вложенности и позволяет быстро оценить организацию файлов.

Важно отметить, что функция display_tree не сохраняет состояние выделения элементов, так как её основная цель – информационный просмотр, а не активная навигация. Однако пользователь может в любой момент нажать t для возврата к обычному режиму со списком и курсором. Переключение режимов происходит мгновенно, что делает взаимодействие удобным.

4.3 Описание реализации файловых операций

Все операции над файлами и каталогами реализованы в виде отдельных функций, вызываемых из handle_input. Каждая операция оперирует полными путями, которые формируются на основе current_path и имени выбранного элемента. Такой подход обеспечивает однозначную идентификацию объектов и корректную работу даже при наличии символических ссылок или специальных символов в именах.

Копирование осуществляется функцией copy_item. В ней формируется командная строка cp -r "from" "to", которая затем передаётся в system. Использование cp -r гарантирует рекурсивное копирование для каталогов, а также корректную обработку файлов. Перед выполнением команды путь назначения to может быть преобразован: если он не начинается с /, то считается относительным и дополняется current_path. После выполнения проверяется код возврата system: ненулевое значение свидетельствует об ошибке, и пользователю выводится сообщение с просьбой нажать любую клавишу для продолжения.

Перемещение реализовано с помощью системного вызова rename, который работает атомарно и более эффективно, чем копирование с последующим удалением. В функции move_item сначала проверяется, является ли путь назначения существующим каталогом с помощью is_directory. Если да, то к этому пути добавляется имя исходного объекта, полученное через get_filename. Таким образом, если пользователь указывает в качестве назначения каталог, объект перемещается внутрь этого каталога, сохраняя своё имя.

Системная команда rm -rf, которая рекурсивно удаляет всё содержимое. Перед выполнением удаления программа запрашивает подтверждение: выводится сообщение Are you sure you want to delete ...? (y/n):, и ожидается нажатие клавиши y или n. Если пользователь подтверждает, выполняется соответствующая операция. После удаления индекс выбранного элемента корректируется: если удалённый элемент был последним в списке, selected_index уменьшается на единицу, чтобы курсор не оказался за пределами списка.

Создание нового каталога использует mkdir с правами доступа 0777 (полные права для владельца, группы и всех остальных). После создания новый объект автоматически появится в списке при следующем обновлении интерфейса, поскольку список перечитывается перед каждым выводом.

Просмотр файла реализован в функции view_file. Сначала с помощью stat извлекаются метаданные файла: права доступа, владелец, группа, размер и время последней модификации. Права доступа выводятся в символьном формате (например, -rw-r--r--). Имена владельца и группы получаются через getpwuid и getgrgid. Затем файл открывается для чтения, и его содержимое построчно выводится на экран. После завершения вывода программа ожидает нажатия любой клавиши, чтобы вернуться к основному интерфейсу.

Все файловые операции обрабатывают ошибки с помощью вывода соответствующих сообщений и ожидания нажатия клавиши, что предотвращает внезапный выход из программы и позволяет пользователю ознакомиться с причиной проблемы.

4.4 Тестирование программы

Для проверки надёжности и производительности файлового менеджера были реализованы два специальных режима тестирования, запускаемых через

аргументы командной строки. Соответствующий код размещён в функции main перед основным циклом обработки ввода. Тестирование позволяет автоматизировать проверку ключевых функций и измерить время выполнения массовых операций.

Тестирование производительности предназначено для оценки скорости создания и удаления большого количества файлов. При запуске с этим аргументом программа создаёт временный каталог test_dir (если он уже существует, используется существующий). Затем в цикле от 0 до 99999 создаются файлы с именами test_file_0 ... test_file_99999. Для измерения времени используются часы высокого разрешения из библиотеки <chrono>. На рисунке 4.9 зафиксированы результаты выполнения: создание 100 000 файлов заняло 4.21078 секунды. После этого программа удаляет каталог test_dir со всем содержимым с помощью системной команды rm -rf, что заняло 1.92385 секунды. Общее время выполнения теста составило 6.13467 секунды. Эти показатели демонстрируют, что программа способна эффективно обрабатывать большое количество объектов, а используемые системные вызовы обеспечивают приемлемую производительность даже при высокой нагрузке.

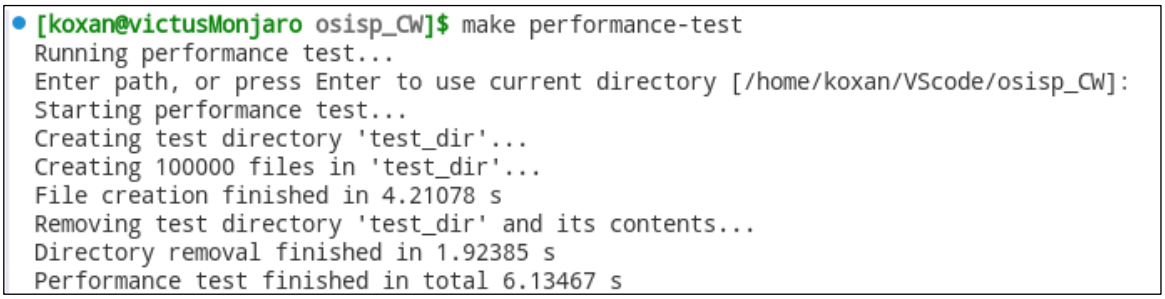


Рисунок 4.9 – Тестирование производительности

Тестирование корректности (correctness_test) проверяет правильность выполнения базовых операций в строгой последовательности. Тест состоит из семи шагов, каждый из которых сопровождается выводом статуса:

1 Создание тестового каталога test_dir с помощью mkdir. Успешное создание подтверждается сообщением Directory created successfully.

2 Создание файла test_dir/test_file.txt и запись в него строки hello world from correctness test. Файл создаётся через std::ofstream, содержимое записывается, затем файл закрывается.

3 Копирование файла в test_dir/test_file_copy.txt с использованием команды cp -r. Проверяется код возврата system.

4 Проверка содержимого скопированного файла: он открывается, считывается первая строка и сравнивается с исходной. При совпадении выводится Copied file content verified successfully.

5 Перемещение исходного файла в test_dir/test_file_moved.txt с помощью rename. Успешность проверяется по коду возврата.

6 Проверка перемещения: открывается файл по новому пути для подтверждения существования, и проверяется, что файл по старому пути больше не открывается. Выводятся соответствующие сообщения.

7 Очистка тестового каталога с помощью rm -rf. Если все шаги выполнены успешно, выводится Correctness test PASSED.

На рисунке 4.10 виден весь процесс выполнения теста. Каждый шаг сопровождается детальным выводом, что позволяет легко локализовать возможные сбои. Пройденный тест подтверждает корректность реализации копирования, перемещения, удаления и создания объектов, а также надёжность проверки состояния файловой системы после операций.

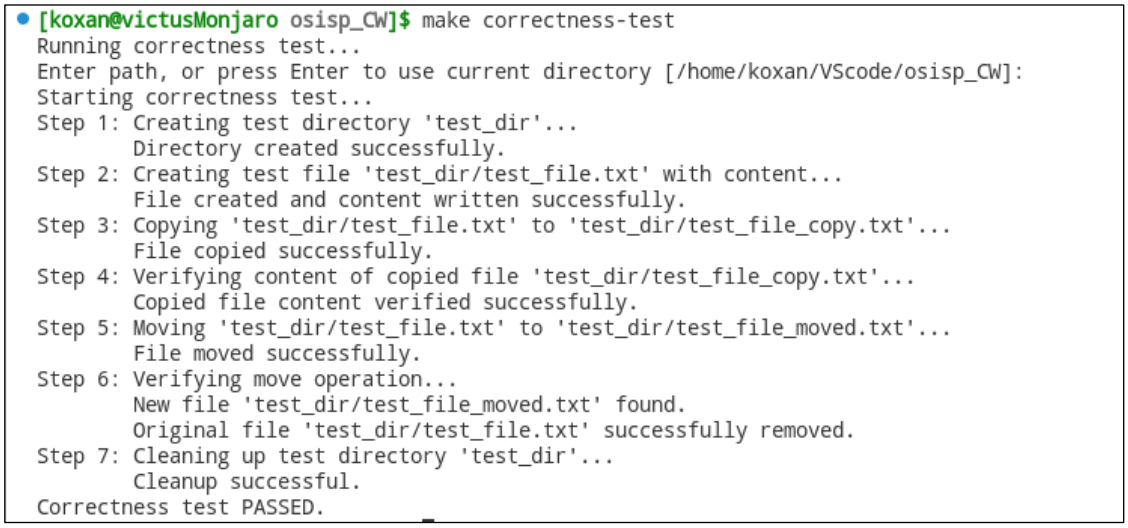


Рисунок 4.10 – Тестирование корректности выполнения

5 АРХИТЕКТУРА РАЗРАБАТЫВАЕМОЙ ПРОГРАММЫ

5.1 Общая структура программы

Программа представляет собой консольный файловый менеджер, реализованный на языке C++ с использованием стандартных библиотек и системных вызовов POSIX. Архитектура построена по принципу разделения ответственности: основные функции вынесены в отдельные подпрограммы, что обеспечивает модульность и удобство сопровождения. Весь исходный код сосредоточен в одном файле main.cpp, что упрощает сборку и распространение. Для компиляции используется Makefile, наполнение которого представлено на рисунке 5.1.

```
--- Metadata for Makefile ---
Permissions: -rw-r--r--
Owner: koxan
Group: koxan
Size: 510 bytes
Last modified: Sat Feb 14 23:09:36 2026
-----------------------------------

CXX = g++
CXXFLAGS = -std=c++20 -Wall -Wextra -pedantic

SRC = main.cpp
TARGET = file_manager

.PHONY: all clean performance-test correctness-test

all: $(TARGET)

$(TARGET): $(SRC)
        $(CXX) $(CXXFLAGS) -o $(TARGET) $(SRC)

clean:
        rm -f $(TARGET) *.o

performance-test: all
        @echo "Running performance test..."
        @./$(TARGET) performance_test

correctness-test: all
        @echo "Running correctness test..."
        @./$(TARGET) correctness_test
        @echo "Correctness test finished."
        @rm -rf test_dir

run: all
        ./$(TARGET)
```

Рисунок 5.1 – Наполнение Makefile

В начале файла main.cpp объявлены все используемые заголовочные файлы, обеспечивающие работу с файловой системой (<dirent.h>, <sys/stat.h>, <unistd.h>), управлением терминалом (<termios.h>), стандартными потоками (<iostream>, <fstream>), обработкой строк (<string>, <cstring>), алгоритмами (<algorithm>), временем (<chrono>, <ctime>), а также с информацией о

пользователях и группах (<pwd.h>, <grp.h>). Такое разнообразие включённых библиотек обусловлено необходимостью выполнения различных задач: от простого вывода списка файлов до получения расширенных метаданных.

Далее следуют прототипы всех пользовательских функций, что позволяет вызывать их в любом порядке, не заботясь о порядке определения. Это важный элемент организации кода, особенно при наличии рекурсивных вызовов (например, display_tree вызывает саму себя). Глобальная переменная tree_view_enabled хранит текущее состояние режима отображения и используется для переключения между стандартным списком и древовидным представлением.

Функция main выполняет несколько ключевых задач. Сначала она получает текущий рабочий каталог с помощью getcwd и выводит приглашение для ввода начального пути. В отличие от предыдущих версий, теперь ввод пути выполняется в цикле до тех пор, пока не будет введён корректный каталог – это улучшает пользовательский опыт, позволяя исправить ошибку без перезапуска. Затем main проверяет аргументы командной строки: если передан performance_test или correctness_test, запускается соответствующий тестовый режим, который полностью изолирован от основного цикла. В противном случае программа входит в бесконечный цикл, который отвечает за отрисовку интерфейса и обработку ввода. В каждой итерации цикла в зависимости от значения tree_view_enabled вызывается либо display_directory (режим списка с курсором), либо display_tree (режим дерева). После отрисовки вызывается handle_input, которая обрабатывает нажатие клавиши и изменяет состояние программы (текущий путь, индекс выбора, флаг режима). Такой цикл обеспечивает реактивность интерфейса и постоянное обновление экрана. Исходный код программы представлен в приложении А.

5.2 Описание функциональной схемы навигатора

Навигация по файловой системе реализована через две основные функции: display_directory и handle_input. Первая отвечает за визуализацию списка файлов и каталогов в текущей директории, вторая – за изменение состояния на основе пользовательского ввода.

Функция display_directory принимает путь к текущей директории и индекс выбранного элемента. Она очищает экран с помощью system("clear"), выводит заголовок с путём и подсказку по управлению. Затем открывает директорию через opendir, читает все записи с помощью readdir, исключая . и .., и сохраняет имена в вектор entries. После закрытия директории вектор сортируется в алфавитном порядке с помощью std::sort. Далее в цикле для каждого элемента выводится либо префикс > (если индекс совпадает), либо два пробела, а затем имя элемента. Такой подход обеспечивает классический интерфейс с перемещающимся курсором.

Функция handle_input является ядром навигации. Она получает текущий путь и индекс выбранного элемента по ссылке, чтобы изменять их, а также флаг режима дерева. В начале она считывает содержимое текущей директории

аналогично display_directory, чтобы иметь актуальный список entries для операций, которые могут зависеть от выделенного элемента. Затем с помощью getch считывается один символ, и выполняется ветвление по коду клавиши.

Навигационные клавиши w и s изменяют selected_index в пределах от 0 до entries.size() - 1, но только если режим дерева не активен (поскольку в режиме дерева нет курсора). Клавиша f реализует поиск: запрашивает имя у пользователя, находит его в векторе с помощью std::find и перемещает курсор на найденный элемент или выводит сообщение об ошибке. Клавиша Enter обрабатывает открытие объекта: если выбранный элемент является каталогом, то current_path обновляется путём добавления имени подкаталога, а selected_index сбрасывается в 0; если файл, вызывается view_file. Клавиша b осуществляет переход на уровень выше: с помощью get_parent_path получается родительский путь, и если он отличается от текущего, то current_path обновляется, а selected_index сбрасывается. Особый случай – переход из корня, где get_parent_path может вернуть пустую строку; тогда используется явная установка "/".

Таким образом, навигатор поддерживает как вертикальное перемещение внутри текущего каталога, так и горизонтальное (переход между каталогами), а также поиск и возврат назад. Все операции визуально отражаются на экране благодаря перерисовке после каждого ввода.

5.3 Описание блок схемы обработки событий ввода и перерисовки

Алгоритм обработки событий ввода и перерисовки экрана можно представить в виде непрерывного цикла, который выполняется в функции main после инициализации. На каждом шаге цикла выполняются следующие этапы:

1 Определение режима отображения. Проверяется глобальная переменная tree_view_enabled. Если true, вызывается display_tree, которая рекурсивно выводит структуру каталогов, начиная с current_path. Если false, вызывается display_directory, которая выводит список файлов с курсором, используя selected_index.

2 Ожидание ввода. После отрисовки вызывается handle_input, которая блокируется в вызове getch до нажатия клавиши. getch настраивает терминал в неканонический режим, считывает один символ и восстанавливает настройки.

3 Обработка ввода. В handle_input на основе считанного символа изменяются переменные состояния (current_path, selected_index, tree_view_enabled) или выполняются файловые операции (копирование, перемещение, удаление, создание, просмотр). При выполнении операций, которые могут изменить содержимое файловой системы (например, удаление), программа может запросить дополнительный ввод (подтверждение, путь назначения).

4 Цикл повторяется. Поскольку экран очищается при каждом вызове display_directory или display_tree, а состояние изменено, пользователь видит

актуальный интерфейс.

Важной особенностью является то, что все операции чтения списка файлов выполняются заново перед каждым выводом и перед каждой обработкой ввода. Это гарантирует, что изменения, внесённые внешними процессами или самой программой, сразу отражаются в интерфейсе. Такой подход упрощает логику, так как не требуется поддерживать внутренний кэш.

Блок-схема этого алгоритма линейна: начальная инициализация → основной цикл (отрисовка → ввод → обработка) → завершение по нажатию q. Обработка ввода разветвляется в зависимости от клавиши, но в любом случае возвращается в начало цикла. Такая структура характерна для многих консольных приложений с интерактивным интерфейсом и позволяет легко добавлять новые команды, расширяя switch в handle_input.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта был разработан консольный навигатор по файловой системе с поддержкой древовидного отображения каталогов, ориентированный на работу в среде Linux. Программа реализована на языке C++ с использованием стандартных библиотек и системных вызовов POSIX, что обеспечивает высокую производительность, минимальную зависимость от внешних компонентов и полную переносимость в рамках Unix-подобных операционных систем. В процессе проектирования проведён анализ архитектуры TUI-приложений и особенностей файловой системы Unix, включая механизмы индексных дескрипторов, права доступа и системные вызовы, что позволило обоснованно выбрать модель представления файловой системы в памяти и определить набор необходимых операций. Изучены платформенные средства управления терминалом и системные API; для обработки клавиатурного ввода реализована собственная функция с переключением терминала в неканонический режим, обеспечивающая мгновенную реакцию на нажатия без необходимости подтверждения клавишей Enter. От использования полноценной библиотеки ncurses решено отказаться в пользу прямого управления терминалом, что сократило размер программы и устранило дополнительные зависимости. Реализована полная навигация по файловой системе (перемещение вверх/вниз, вход в каталоги, возврат на уровень выше), а также набор базовых файловых операций: копирование, перемещение, удаление, создание файлов и каталогов, просмотр содержимого файлов с выводом метаданных. Добавлена функция быстрого поиска файла по имени в текущем каталоге. Проведено тестирование программы в двух автоматизированных режимах. Тест производительности показал, что создание 100 000 пустых файлов занимает около 4,2 секунды, а их рекурсивное удаление около 1,9 секунды, что подтверждает эффективность реализации и адекватность использования системных вызовов. Тест корректности последовательно проверил создание каталога, создание и запись файла, копирование с верификацией содержимого, перемещение и удаление; все шаги теста были успешно пройдены. Обеспечена обработка ошибок, связанных с отсутствием прав доступа, несуществующими путями, попытками удаления непустых каталогов и другими типичными ситуациями; пользователю выводятся информативные сообщения с предложением продолжить работу. Таким образом, цель курсового проекта – разработка интерактивного консольного навигатора с древовидным отображением каталогов и поддержкой базовых файловых операций – достигнута полностью.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

[1] Введение в Midnight Commander [Электронный ресурс]. – Режим доступа: https://midnight-commander.org/. – Дата доступа: 22.02.2026.

[2] Обзор-сравнение TUI файловых менеджеров: mc, ranger, vifm [Электронный ресурс]. – Режим доступа: https://pingvinus.ru/note/compare-mc-ranger-vifm?ysclid=mlxnhs6u2y341115201. – Дата доступа: 22.02.2026.

[3] Сигналы в Unix системах [Электронный ресурс]. – Режим доступа: https://www.kryukov.biz/soderzhanie/protsessy/signaly/. – Дата доступа: 22.02.2026.

[4] Файловая система ОС Unix [Электронный ресурс]. – https://www.inp.nsk.su/~bolkhov/teach/inpunix/start_fs.ru.html. – Дата доступа: 22.02.2026.

[5] Индексные дескрипторы файлов [Электронный ресурс]. – Режим доступа: https://rus-linux.net/kos.php?name=/book1/gl-16/gl_16_03.html. – Дата доступа: 22.02.2026.

[6] Системные вызовы к файловой системе Linux [Электронный ресурс]. – Режим доступа: https://manpages.ubuntu.com/manpages/noble/ru/man2/syscalls.2.html. – Дата доступа: 22.02.2026.

[7] Файловые системы в Linux: их структура и типы [Электронный ресурс]. – Режим доступа: https://timeweb.com/ru/community/articles/struktura-i-tipy-faylovyh-sistem-v-linux. – Дата доступа: 22.02.2026.

[8] Терминал канонический и неканонический [Электронный ресурс]. – Режим доступа: https://rus-linux.net/nlib.php?name=/MyLDP/BOOKS/Linux-tools/10/ltfwp-10-22.html. – Дата доступа: 22.02.2026.

[9] Norton Commander в MS-DOS [Электронный ресурс]. – Режим доступа: http://old-dos.ru/files/file_1458.html. – Дата доступа: 24.03.2026.

[10] Права доступа к файлам и каталогам в Linux. Категории пользователей [Электронный ресурс]. – Режим доступа: https://younglinux.info/bash/rwx. – Дата доступа: 24.03.2026.

[11] Списки управления доступом. Списки DACL и SACL в Windows [Электронный ресурс]. – Режим доступа: https://learn.microsoft.com/ru-ru/windows/win32/secauthz/access-control-lists. – Дата доступа: 24.03.2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <sys/stat.h>
#include <unistd.h>
#include <termios.h>
#include <fstream>
#include <cstring>
#include <chrono>
#include <dirent.h>
#include <iomanip>
#include <ctime>
#include <pwd.h>
#include <grp.h>

void display_directory(const std::string& path, int selected_index);
void display_tree(const std::string& path, int level);
void  handle_input(std::string&  current_path,  int&  selected_index,  bool&
tree_view_enabled);
char getch();
void view_file(const std::string& file_path);
void copy_item(const std::string& from, const std::string& to);
void move_item(const std::string& from, const std::string& to);
void create_new_directory(const std::string& path);
void create_new_file(const std::string& path);
bool is_directory(const std::string& path);
std::string get_filename(const std::string& path);
std::string get_parent_path(const std::string& path);

bool tree_view_enabled = false;

int main(int argc, char *argv[]) {
    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd() error");
        return 1;
    }

    std::string current_path;
    while (true) {
        std::cout << "Enter path, or press Enter to use current directory ["
<< cwd << "]: ";
        std::string path_input;
        std::getline(std::cin, path_input);

        current_path = path_input.empty() ? cwd : path_input;

        if (is_directory(current_path)) {
            break;
        } else {
            std::cerr  <<  "Error:  Invalid  path  or  not  a  directory:  "  <<
current_path << std::endl;
        }
    }

    int selected_index = 0;
```

```
    if (argc > 1) {
        std::string arg = argv[1];
        if (arg == "performance_test") {
            const int num_files = 100000;
            std::cout << "Starting performance test..." << std::endl;

            auto start_create = std::chrono::high_resolution_clock::now();
            std::cout << "Creating test directory 'test_dir'..." << std::endl;
            if (mkdir("test_dir", 0777) != 0 && errno != EEXIST) {
                std::cerr << "Performance test FAILED: could not create
directory 'test_dir'" << std::endl;
                return 1;
            }
            std::cout << "Creating " << num_files << " files in 'test_dir'..."
<< std::endl;
            for (int i = 0; i < num_files; ++i) {
                std::ofstream("test_dir/test_file_" + std::to_string(i));
            }
            auto end_create = std::chrono::high_resolution_clock::now();
            std::chrono::duration<double> diff_create = end_create -
start_create;
            std::cout << "File creation finished in " << diff_create.count() <<
" s" << std::endl;

            std::cout << "Removing test directory 'test_dir' and its
contents..." << std::endl;
            auto start_remove = std::chrono::high_resolution_clock::now();
            if (system("rm -rf test_dir") != 0) {
                std::cerr << "Performance test FAILED: cleanup failed." <<
std::endl;
                return 1;
            }
            auto end_remove = std::chrono::high_resolution_clock::now();
            std::chrono::duration<double> diff_remove = end_remove -
start_remove;
            std::cout << "Directory removal finished in " <<
diff_remove.count() << " s" << std::endl;

            std::chrono::duration<double> total_diff = end_remove -
start_create;
            std::cout << "Performance test finished in total " <<
total_diff.count() << " s\n";
            return 0;
        } else if (arg == "correctness_test") {
            std::cout << "Starting correctness test..." << std::endl;
            const char* test_dir = "test_dir";

            // 1. Create directory
            std::cout << "Step 1: Creating test directory '" << test_dir <<
"'..." << std::endl;
            if (mkdir(test_dir, 0777) != 0 && errno != EEXIST) {
                std::cerr << "Correctness test FAILED: could not create
directory '" << test_dir << "'" << std::endl;
                return 1;
            }
            std::cout << "       Directory created successfully." << std::endl;

            // 2. Create file with content
            std::string test_filename = std::string(test_dir) +
"/test_file.txt";
            std::string test_content = "hello world from correctness test";
            std::cout << "Step 2: Creating test file '" << test_filename << "'
with content..." << std::endl;
            std::ofstream test_file(test_filename);
```

```
            if (!test_file.is_open()) {
                std::cerr << "Correctness test FAILED: could not create file '"
<< test_filename << "'" << std::endl;
                system(("rm -rf " + std::string(test_dir)).c_str());
                return 1;
            }
            test_file << test_content;
            test_file.close();
            std::cout << "       File created and content written successfully."
<< std::endl;

            // 3. Copy file
            std::string     copy_filename     =     std::string(test_dir)     +
"/test_file_copy.txt";
            std::string copy_command = "cp -r \"" + test_filename + "\" \"" +
copy_filename + "\"";
            std::cout << "Step 3: Copying '" << test_filename << "' to '" <<
copy_filename << "'..." << std::endl;
            if(system(copy_command.c_str()) != 0){
                std::cerr << "Correctness test FAILED: copy command failed." <<
std::endl;
                system(("rm -rf " + std::string(test_dir)).c_str());
                return 1;
            }
            std::cout << "        File copied successfully." << std::endl;


            // 4. Verify copy
            std::cout  <<  "Step  4:  Verifying  content  of  copied  file  '"  <<
copy_filename << "'..." << std::endl;
            std::ifstream copy_file(copy_filename);
            if (!copy_file.is_open()) {
                std::cerr << "Correctness test FAILED: could not open copied
file '" << copy_filename << "'" << std::endl;
                system(("rm -rf " + std::string(test_dir)).c_str());
                return 1;
            }
            std::string copy_content;
            std::getline(copy_file, copy_content);
            copy_file.close();
            if (copy_content != test_content) {
                std::cerr << "Correctness test FAILED: content of copied file
is incorrect. Expected '" << test_content << "', got '" << copy_content << "'"
<< std::endl;
                system(("rm -rf " + std::string(test_dir)).c_str());
                return 1;
            }
            std::cout << "         Copied file content verified successfully."
<< std::endl;

            // 5. Move file
            std::string     moved_filename     =     std::string(test_dir)     +
"/test_file_moved.txt";
            std::cout << "Step 5: Moving '" << test_filename << "' to '" <<
moved_filename << "'..." << std::endl;
            if (rename(test_filename.c_str(), moved_filename.c_str()) != 0) {
                perror("rename");
                std::cerr << "Correctness test FAILED: move operation failed."
<< std::endl;
                system(("rm -rf " + std::string(test_dir)).c_str());
                return 1;
            }
            std::cout << "        File moved successfully." << std::endl;
```

```
            // 6. Verify move - check if new file exists and old file is gone
            std::cout << "Step 6: Verifying move operation..." << std::endl;
            std::ifstream moved_file_check(moved_filename);
            if(!moved_file_check.is_open()){
                std::cerr << "Correctness test FAILED: move operation failed,
moved file not found." << std::endl;
                system(("rm -rf " + std::string(test_dir)).c_str());
                return 1;
            }
            moved_file_check.close();
            std::cout << "        New file '" << moved_filename << "' found."
<< std::endl;

            std::ifstream old_file_check(test_filename);
            if(old_file_check.is_open()){
                std::cerr  <<  "Correctness  test  FAILED:  old  file  '"  <<
test_filename << "' still exists after move." << std::endl;
                system(("rm -rf " + std::string(test_dir)).c_str());
                return 1;
            }
            std::cout << "          Original file '" << test_filename << "'
successfully removed." << std::endl;

            // 7. Cleanup
            std::cout << "Step 7: Cleaning up test directory '" << test_dir <<
"'..." << std::endl;
            if (system(("rm -rf " + std::string(test_dir)).c_str()) != 0) {
                std::cerr  <<  "Correctness  test  FAILED:  cleanup  failed."  <<
std::endl;
                return 1;
            }
            std::cout << "        Cleanup successful." << std::endl;

            std::cout << "Correctness test PASSED." << std::endl;
            return 0;
        }
    }

    while (true) {
        if (tree_view_enabled) {
            system("clear");
            std::cout << "Current directory: " << current_path << std::endl;
            std::cout << "Controls: (t) - toggle tree view, (b) - back, (q) -
quit" << std::endl;
            std::cout    <<    "----------------------------------------"    <<
std::endl;
            display_tree(current_path, 0);
        } else {
            display_directory(current_path, selected_index);
        }
        handle_input(current_path, selected_index, tree_view_enabled);
    }

    return 0;
}

char getch() {
    char buf = 0;
    struct termios old;
    memset(&old, 0, sizeof(old));
    if (tcgetattr(0, &old) < 0)
        perror("tcsetattr()");
    old.c_lflag &= ~ICANON;
    old.c_lflag &= ~ECHO;
```

```
    old.c_cc[VMIN] = 1;
    old.c_cc[VTIME] = 0;
    if (tcsetattr(0, TCSANOW, &old) < 0)
        perror("tcsetattr ICANON");
    if (read(0, &buf, 1) < 0)
        perror("read()");
    old.c_lflag |= ICANON;
    old.c_lflag |= ECHO;
    if (tcsetattr(0, TCSADRAIN, &old) < 0)
        perror("tcsetattr ~ICANON");
    return buf;
}

bool is_directory(const std::string& path) {
    struct stat st;
    if (stat(path.c_str(), &st) == 0) {
        return S_ISDIR(st.st_mode);
    }
    return false;
}

std::string get_filename(const std::string& path) {
    size_t pos = path.find_last_of("/");
    if (pos != std::string::npos) {
        return path.substr(pos + 1);
    }
    return path;
}

std::string get_parent_path(const std::string& path) {
    size_t pos = path.find_last_of("/");
    if (pos != std::string::npos && pos > 0) {
        return path.substr(0, pos);
    }
    if (pos == 0 && path.length() > 1) {
        return "/";
    }
    return "";
}


void display_tree(const std::string& path, int level) {
    DIR *dir;
    struct dirent *entry;

    if (!(dir = opendir(path.c_str()))) {
        perror("opendir");
        return;
    }

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") ==
0) {
            continue;
        }

        for (int i = 0; i < level; ++i) {
            std::cout << "  ";
        }
        std::cout << "|-- " << entry->d_name << std::endl;

        std::string entry_path = path + "/" + entry->d_name;
        if (is_directory(entry_path)) {
            display_tree(entry_path, level + 1);
```

```
        }
    }
    closedir(dir);
}

void display_directory(const std::string& path, int selected_index) {
    system("clear");
    std::cout << "Current directory: " << path << std::endl;
    std::cout << "Controls: (w/s) - up/down, (f) - find, (enter) - open, (b) -
back, (c) - copy, (m) - move, (d) - delete, (n) - new file, (N) - new directory,
(t) - tree view, (q) - quit" << std::endl;
    std::cout << "----------------------------------------" << std::endl;

    std::vector<std::string> entries;
    DIR *dir;
    struct dirent *entry;

    if (!(dir = opendir(path.c_str()))) {
        perror("opendir");
        getch();
        return;
    }

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") != 0 && strcmp(entry->d_name, "..") !=
0) {
            entries.push_back(entry->d_name);
        }
    }
    closedir(dir);
    std::sort(entries.begin(), entries.end());


    for (size_t i = 0; i < entries.size(); ++i) {
        if (i == static_cast<size_t>(selected_index)) {
            std::cout << "> ";
        } else {
            std::cout << "  ";
        }
        std::cout << entries[i] << std::endl;
    }
}

void handle_input(std::string& current_path, int& selected_index, bool&
tree_view_enabled) {
    char c = getch();
    std::vector<std::string> entries;
    DIR *dir;
    struct dirent *entry;

    if (!(dir = opendir(current_path.c_str()))) {
        perror("opendir");
        getch();
        return;
    }

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") != 0 && strcmp(entry->d_name, "..") !=
0) {
            entries.push_back(entry->d_name);
        }
    }
    closedir(dir);
    std::sort(entries.begin(), entries.end());
```

```
    switch (c) {
        case 't': // Tree
            tree_view_enabled = !tree_view_enabled;
            break;
        case 'w': // Up
            if (!tree_view_enabled && selected_index > 0) {
                selected_index--;
            }
            break;
        case 's': // Down
            if       (!tree_view_enabled       &&       selected_index       <
static_cast<int>(entries.size()) - 1) {
                selected_index++;
            }
            break;
        case 'f': // Find
            if (!tree_view_enabled) {
                std::cout << "Enter name to find: ";
                std::string name_to_find;
                std::cin >> name_to_find;

                auto    it    =    std::find(entries.begin(),    entries.end(),
name_to_find);
                if (it != entries.end()) {
                    selected_index = std::distance(entries.begin(), it);
                } else {
                    std::cout << "\n'" << name_to_find << "' not found. Press
any key to continue." << std::flush;
                    getch();
                }
            }
            break;
        case '\n': // Enter
            if (!tree_view_enabled && !entries.empty()) {
                std::string   selected_path   =   current_path   +   "/"   +
entries[selected_index];
                if (is_directory(selected_path)) {
                    current_path = selected_path;
                    selected_index = 0;
                } else {
                    view_file(selected_path);
                }
            }
            break;
        case 'b': // Back
        {
            std::string parent_path = get_parent_path(current_path);
            if (!parent_path.empty() && parent_path != current_path) {
                current_path = parent_path;
                selected_index = 0;
            } else if (current_path != "/") {
                 current_path = "/";
                 selected_index = 0;
            }
        }
            break;
        case 'c': // Copy
            if (!tree_view_enabled && !entries.empty()) {
                std::string    from    =    current_path    +    "/"    +
entries[selected_index];
                std::cout << "Enter destination path: ";
                std::string to_str;
```

```
                std::cin >> to_str;

                std::string final_to_path = to_str;
                if (!to_str.empty() && to_str[0] != '/') {
                    final_to_path = current_path + "/" + to_str;
                }
                copy_item(from, final_to_path);
            }
            break;
        case 'm': // Move
            if (!tree_view_enabled && !entries.empty()) {
                std::string from = current_path + "/" +
entries[selected_index];
                std::cout << "Enter destination path: ";
                std::string to_str;
                std::cin >> to_str;

                std::string final_to_path = to_str;
                if (!to_str.empty() && to_str[0] != '/') {
                    final_to_path = current_path + "/" + to_str;
                }
                move_item(from, final_to_path);
            }
            break;
        case 'd': // Delete
            if (!tree_view_enabled && !entries.empty()) {
                std::string path_to_delete = current_path + "/" +
entries[selected_index];
                std::cout << "Are you sure you want to delete " <<
get_filename(path_to_delete) << "? (y/n): ";
                std::cout << std::flush;
                char choice = getch();
                if (choice == 'y') {
                    if (is_directory(path_to_delete)) {
                        std::string command = "rm -rf \"" + path_to_delete +
"\"";
                        system(command.c_str());
                    } else {
                        remove(path_to_delete.c_str());
                    }
                    if (selected_index > 0 && selected_index >=
static_cast<int>(entries.size()) - 1) {
                        selected_index--;
                    }
                }
            }
            break;
        case 'n':
            if (!tree_view_enabled) {
                std::cout << "Enter new file name: ";
                std::string name;
                std::cin >> name;
                create_new_file(current_path + "/" + name);
            }
            break;
        case 'N':
            if (!tree_view_enabled) {
                std::cout << "Enter new directory name: ";
                std::string name;
                std::cin >> name;
                create_new_directory(current_path + "/" + name);
            }
            break;
        case 'q':
```

```
            exit(0);
            break;
    }
}

void view_file(const std::string& file_path) {
    system("clear");

    struct stat st;
    if (stat(file_path.c_str(), &st) == 0) {
        std::cout << "--- Metadata for " << get_filename(file_path) << " ---"
<< std::endl;

        std::cout << "Permissions: ";
        std::cout << (S_ISDIR(st.st_mode) ? 'd' : '-');
        std::cout << ((st.st_mode & S_IRUSR) ? 'r' : '-');
        std::cout << ((st.st_mode & S_IWUSR) ? 'w' : '-');
        std::cout << ((st.st_mode & S_IXUSR) ? 'x' : '-');
        std::cout << ((st.st_mode & S_IRGRP) ? 'r' : '-');
        std::cout << ((st.st_mode & S_IWGRP) ? 'w' : '-');
        std::cout << ((st.st_mode & S_IXGRP) ? 'x' : '-');
        std::cout << ((st.st_mode & S_IROTH) ? 'r' : '-');
        std::cout << ((st.st_mode & S_IWOTH) ? 'w' : '-');
        std::cout << ((st.st_mode & S_IXOTH) ? 'x' : '-') << std::endl;

        struct passwd *pw = getpwuid(st.st_uid);
        struct group  *gr = getgrgid(st.st_gid);
        if (pw) std::cout << "Owner: " << pw->pw_name << std::endl;
        if (gr) std::cout << "Group: " << gr->gr_name << std::endl;

        std::cout << "Size: " << st.st_size << " bytes" << std::endl;

        char time_buf[80];
        time_t mtime = st.st_mtime;
        strftime(time_buf, sizeof(time_buf), "%c", localtime(&mtime));
        std::cout << "Last modified: " << time_buf << std::endl;

        std::cout << "-----------------------------------" << std::endl <<
std::endl;
    }

    std::ifstream file(file_path);
    if (file.is_open()) {
        std::string line;
        while (std::getline(file, line)) {
            std::cout << line << std::endl;
        }
        file.close();
        std::cout << "\nPress any key to continue...";
        getch();
    } else {
        perror("open");
        std::cout << "\nPress any key to continue...";
        getch();
    }
}

void copy_item(const std::string& from, const std::string& to) {
    std::string command = "cp -r \"" + from + "\" \"" + to + "\"";
    int result = system(command.c_str());
    if (result != 0) {
        std::cerr << "Error copying item. Command failed: " << command <<
std::endl;
        std::cout << "\nPress any key to continue...";
```

```
        getch();
    }
}

void move_item(const std::string& from, const std::string& to) {
    std::string final_to_path = to;
    if (is_directory(to)) {
        final_to_path = to + "/" + get_filename(from);
    }

    if (rename(from.c_str(), final_to_path.c_str()) != 0) {
        perror("rename");
        std::cout << "\nPress any key to continue...";
        getch();
    }
}

void create_new_directory(const std::string& path) {
    if (mkdir(path.c_str(), 0777) != 0) {
        perror("mkdir");
        std::cout << "\nPress any key to continue...";
        getch();
    }
}

void create_new_file(const std::string& path) {
    std::ofstream file(path);
    if (!file.is_open()) {
        perror("create file");
        std::cout << "\nPress any key to continue...";
        getch();
    }
}
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Функциональная схема навигатора

ПРИЛОЖЕНИЕ В
(обязательное)
Блок схема алгоритма, обработки событий ввода и перерисовки экрана

ПРИЛОЖЕНИЕ Г
(обязательное)
Графический интерфейс пользователя

ПРИЛОЖЕНИЕ Д
(обязательное)
Ведомость курсового проекта